Results in Engineering 29 (2026) 108982

Contents lists available at ScienceDirect

Results in Engineering

journal homepage: www.sciencedirect.com/journal/results-in-engineering

Research paper

# SFA-MOAHA: A multi-strategy self-learning algorithm for multi-robot path planning in complex environments

Ao Ren [a], Junxing Zhang [a,*], Jiaxue Ran [b], Jiewen Pang [a], Lunjie Wang [a]

[a] Key Laboratory of Advanced Manufacturing Technology, Ministry of Education, Guizhou University, Guiyang 550025, China
[b] School of Mechanical Engineering, Guizhou University, Guiyang 550025, China

ARTICLE INFO

*Keywords:*
Multi-robot path planning
Multi-objective artificial hummingbird algorithm
DQN
Spiral approximation
Forward fusion learning

ABSTRACT

Multi-robot systems are crucial in intelligent manufacturing. In dynamic environments, they face complex tasks requiring efficient coordination, with the multi-robot path planning (MRPP) problems being particularly significant. The current algorithms for MRPP problems face difficulties in dealing with dynamic uncertainties and real-time adaptability. To tackle the MRPP problem, this study proposes a spiral forward-adaptability multi-objective artificial hummingbird algorithm (SFA-MOAHA). It combines a spiral approximation strategy and a forward fusion learning strategy to boost exploration and exploitation, and integrates Deep Q-Network (DQN)to adaptively select the best search schemes based on iterative feedback. A multitude of simulation experiments have been carried out to validate the efficacy of the SFA-MOAHA. The experiments cover tests with three benchmark test sets (ZDT, DTLZ, and WFG), engineering design problems, and solving MRPP problems with different robot numbers and map complexities. Compared to traditional algorithms, SFA-MOAHA shows superiority in obtaining well-distributed and converged solutions for the benchmark tests and engineering problems. In two environments of different complexities, SFA-MOAHA shortens the path lengths of 4 mobile robots by 6.23% and 7.45% respectively, and those of 8 robots by 7.89% and 9.61% respectively, outperforming multi-objective artificial hummingbird algorithm (MOAHA). This study demonstrates that SFA-MOAHA works excellently in complex situations and offers an adaptive learning solution for real-time multi-robot coordination.

## 1. Introduction

In recent decades, robotics has been increasingly applied in human life and industrial automation such as search-and-rescue [1] and material transportation [2]. Multi-robot systems, prized for their superior collaboration and efficiency, have become indispensable in complex operational scenarios including intelligent warehouses [3], manufacturing factories [4], and oilfield inspections [5]. In these systems, MRPP is crucial for multi-robot systems as it determines movement of robots. The multi-robot need plan a set of adaptable, collision-free paths in complex environments by solving a multi-objective optimization problem (MOOP) [6]. The MRPP aims to plan such paths for multiple robots to reach their destinations safely and efficiently. Therefore, formulating a MOOP to solve MRPP problem in continuously changing complex environments is a considerable challenge [7]. This challenge arises from several interrelated factors, including the need to compute optimal collision-free paths for all robots, account for inherent environmental uncertainties, reconcile conflicting objectives, and manage the coupling effects between robots. Together, these demands impose high computational complexity on the planning process. Given this complexity, classical reactive path planning methods, such as the artificial potential field method [8] and roadmap method [9] often perform inadequately as they struggle with dynamic obstacles. Similarly, graph-based search algorithms, including Dijkstra's method [10] and A* algorithm [11], typically struggle with scalability and real-time performance when applied to the large, interactive state spaces inherent in multi-robot systems. Thereby, constructing effective mathematical formulations and efficiently resolving multi-objective conflicts are crucial for addressing these core challenges in path planning.

To overcome these drawbacks, researchers have resorted to meta-heuristic algorithms [12–14], drawing inspiration from evolutionary processes or intelligent foraging behaviors in nature, with a simple structure and high adaptability [15–18]. Numerous meta-heuristic algorithms have emerged in recent decades to address robot path planning, such as Particle Swarm Optimization [19–22], Genetic Algorithm

* Corresponding author.
*E-mail address:* jx_zhanggz@163.com (J. Zhang).

https://doi.org/10.1016/j.rineng.2026.108982
Received 11 September 2025; Received in revised form 21 December 2025; Accepted 3 January 2026
Available online 6 January 2026
2590-1230/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

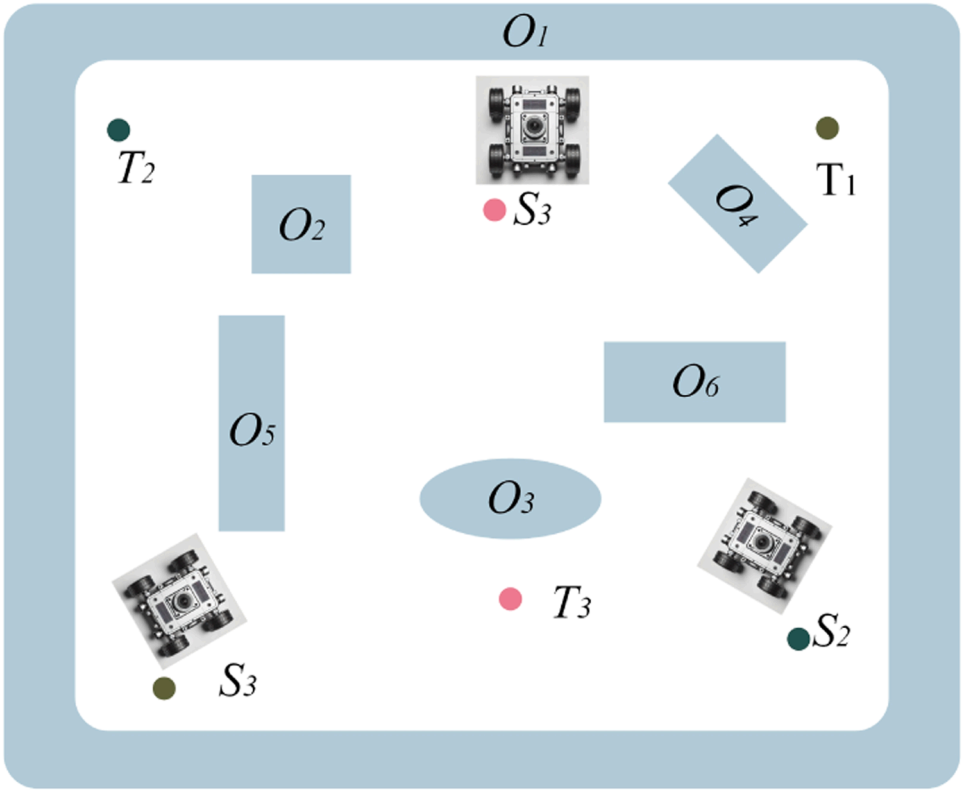



**Fig. 1.** Environment example of MRPP problem.

[23,24], Ant Colony Optimization [25–27], Artificial Bee Colony [28, 29], and other algorithms [30–32]. When path-planning objective functions are well-defined in dynamic environments, meta-heuristic algorithms find optimal solutions faster than classical and graph-based methods.

Besides the aforementioned meta-heuristic algorithms, the artificial hummingbird algorithm (AHA) stands out for optimization problems. Inspired by the foraging behavior of hummingbirds, it mimics their flight and foraging to balance exploration and exploitation. With proper parameter adjustment, AHA can explore effectively and find high-quality solutions [33]. Empirical studies demonstrate the superiority of AHA in solution quality and calculating efficiency compared to numerous competing meta-heuristics across various optimization benchmarks [34]. Given this, Zhao et al. [35] propose a MOAHA for MOOPs by integrating non-dominated sorting and dynamic elimination crowding distance (DECD) strategies. MOAHA and its variants have found broad applications across various fields. For instances, the authors of [36] apply MOAHA to optimize the critical buckling response of spherical shell plates. MOAHA has also been used for the accurate prediction of photovoltaic power [37] and the optimal planning of renewable energy-based distributed generation systems [38]. The extensive multi-objective optimization applications of MOAHA demonstrate its potential for solving MRPP.

It is worth noting that a substantial body of existing research, such as predefined-time consensus control based on dynamic gain [39,40], focuses on the underlying cooperative control theory of multi-agent systems, aiming to ensure the stability and speed of state convergence. However, these methods do not address the high-level path planning problem for multi-robots in complex environments, such as dynamic obstacle avoidance and unstructured map search. In contrast, the research presented in this paper directly targets the specific application problem of MRPP in complex environments.

Although MOAHA addresses a variety of multi-objective optimization problems, it still exhibits inherent limitations when applied to MRPP. Specifically, for complex optimization problems, MOAHA and its variants often exhibit issues such as poor convergence, premature convergence, and weak exploration, primarily due to their similar dimension-updating mechanisms and search-equation structures [34]. Therefore, developing new search strategies is necessary to prevent multiple robots from getting stuck in local optima. On the other hand, their strategies-selection mechanism is not robust. The high randomness leads to significant fluctuations in fitness values and the number of updated individuals per iteration, which may cause operator biases when handling robot collisions. Reinforcement learning learns optimal policies through interactions between agents and environments, making it suitable for enhancing the adaptive capabilities of evolutionary algorithms [41]. It has been applied in multi-objective evolutionary algorithms for parameter selection [42], offspring generation [43] and operator selection [44]. However, these studies treat the entire population as the agent, while the use of individuals as independent agents in the search and selection of multi-objective heuristic algorithms remains unexplored. Moreover, consistent with the “no free lunch” theorem, no single algorithm can perform optimally across all tasks, as different algorithm possess distinct merits [45]. For the real-world MRPP problems that feature large and complex search spaces, the performance of most meta-heuristic algorithms degrades, and their convergence slows considerably. Therefore, improving the MOAHA to better address the challenges of MRPP is of great significance for advancing practical MRPP solutions.

To the best of our knowledge, no existing work has integrated DQN and forward-fusion strategies into MOAHA specifically to overcome its limitations in the MRPP context. This paper proposes a FA-MOAHA to solve MRPP problem. The main contributions are summarized below:

(1) A dynamic spiral approximation is integrated into the omnidirectional flight mode of MOAHA. This mechanism continuously modulates the spiral step size in real time based on individual stagnation duration and total evaluation count. Therefore, the

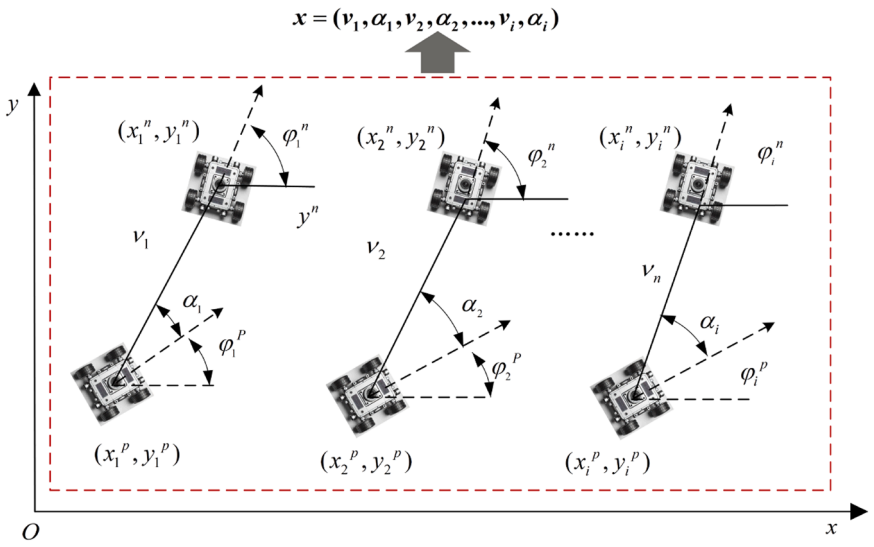



**Fig. 2.** Illustration the position update of the robot.

improved algorithm can selectively intensify local exploitation and refocus its search efforts whenever stuck in a local optimum.

(2) A forward fusion learning mechanism is proposed, which enables solutions in lower-ranked frontiers to learn from randomly selected elites in all superior frontiers. This cross-frontier learning enhances population diversity and reduces premature convergence, which marks a significant improvement over the basic MOAHA that uses only the first pareto front.

(3) A DQN is embedded within the MOAHA to adaptively select search strategies. By utilizing real-time feedback on convergence and diversity, the DQN dynamically balances exploration and exploitation. This policy outperforms traditional fixed-rule methods in both flexibility and effectiveness for the MOAHA [35].

(4) An SFA-MOAHA is proposed for the MRPP problems. It integrates three strategies to enhance adaptability. Leveraging real-time simulation data feedback, it constructs a solution mechanism that generates high-quality solutions in complex scenarios. Compared with traditional methods [46], this algorithm excels in solution quality.

The organization of the next sections is as follows. Section 2 presents the preliminaries and formulation of the MRPP problem. In Section 3, the SFA-MOAHA is proposed and in Section 4, tests and experiments are performed on numerical and engineering design problems. Section 5 employs the SFA-MOAHA to the MRPP problem. Section 6 concludes the article.

## 2. Preliminaries and problem formulation of MRPP

### 2.1. Preliminaries

This paper addresses the issues of MRPP in two-dimensional surroundings with static obstacles. Before formulating the problem, the following assumptions are made:

Assumptions1 [47]. Each robot knows its starting and destination positions and can independently decide the direction and speed of its next move until it reaches the target position or a collision occurs.

Assumptions2 [28]. To make the model more realistic, the robots are regarded as squares.

The robots operate in an environment that consists of both obstacle areas and free spaces, with start and target points placed within the latter. As an example shown in Fig. 1, this environment represents a multi-robot path planning scenario featuring six static obstacles ($O_1$ – $O_6$), three robot start positions ($S_1$ – $S_2$), and three corresponding goal positions ($T_1$ – $T_3$).

Referencing [48], most path planning problems focus on efficiency, safety, and accuracy. In MRPP, the goal is to schedule a collision-free paths for multiple robots from start to target points, avoiding obstacles and other robots. To achieve this, the literature [28] devises an improved ABC algorithm. It calculates a next position of a robot based on the current one. By continuously identifying viable positions, it maps out the path until the robot reaches the target. Early-arriving robots wait for the others.

Based on the above, this paper studies the MRPP problem. In Section 2.3, we will define several objective functions related to optimal and safe to model the problem. These functions are used to shorten path lengths and avoid collisions with obstacles at each step. Then, Section 3 presents the proposed SFA-MOAHA to efficiently solve the formulated MRPP problem.

### 2.2. Robot kinematic model

Considering the change rate of the orientation, the kinematic equations of the robot $i$ $(i = 1, 2, \ldots, n)$ are formulated as [49]:

$$\begin{bmatrix} \dot{x}_i \\ \dot{y}_i \\ \dot{\varphi}_i \end{bmatrix} = \begin{bmatrix} \cos(\varphi_i) & 0 \\ \sin(\varphi_i) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v_i \\ w_i \end{bmatrix}, \tag{1}$$

where $(x_i, y_i)$ stands out the coordinates, $\varphi_i$ represents the orientation, $v_i$ and $w_i$ indicate the linear and angular velocities of the robot $i$, respectively.

**Remark 1.** This work incorporates realistic geometric and kinematic models to better reflect physical constraints, whereas early or simplified approaches often treated robots as kinematic-free mass points [50]. Based on Assumption 2, which models robots as squares, the corresponding model 1 incorporates both translational and rotational motion, offering a more realistic representation.

### 2.3. Displacement representation

Based on the kinematic model above, the displacement of the robot $i$ $(i = 1, 2, \ldots, N)$ at each step can be formulated by its velocity and rotation angle. As shown in Fig. 2, the robot moves from its current to a new position. The representation for this displacement is as follows:

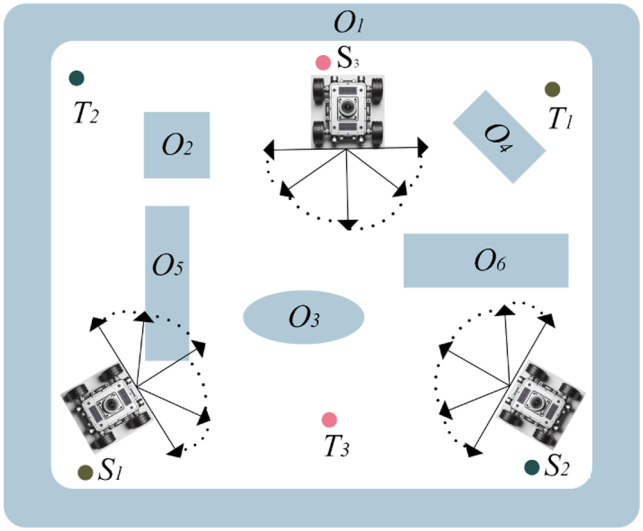



**Fig. 3.** Simulated radar multi-direction front detection diagram.

$$\begin{cases} x_i^{\mathrm{n}} = x_i^p + \cos(\alpha_i + \varphi_i^{\mathrm{p}}) \cdot v_i \cdot \Delta t \\ y_i^{\mathrm{n}} = y_i^{\mathrm{p}} + \sin(\alpha_i + \varphi_i^{\mathrm{p}}) \cdot v_i \cdot \Delta t \end{cases}, \tag{2}$$

where $(x_i^p, y_i^{\mathrm{p}})$ and $(x_i^{\mathrm{n}}, y_i^{\mathrm{n}})$ represent the position at time $t$ and $(t + \Delta t)$, $\alpha_i$ denotes the rotation angle, $\varphi_i^{\mathrm{p}}$ indicates the angle between the heading direction of the robot $i$ and the $x$- axis, respectively.

Set $\Delta t = 1$ and update $\varphi_i^{\mathrm{n}} = \alpha_i + \varphi_i^{\mathrm{p}}$; then, the Eq. (2) can be rewritten as:

$$\begin{cases} x_i^{\mathrm{n}} = x_i^p + \cos\varphi_i^{\mathrm{n}} \cdot v_i \\ y_i^{\mathrm{n}} = y_i^{\mathrm{p}} + \sin\varphi_i^{\mathrm{n}} \cdot v_i \end{cases}, \tag{3}$$

### 2.4. Objective functions of MRPP problem

According to the Section 2.1, three objective functions $f_1$, $f_2$ and $f_3$ are formulated to schedule the optimum safe path for multiple robots. Their purposes are to minimize path lengths, prevent robot-to-robot collisions, and avoid step-by-step collisions with obstacles. Specifically, $f_1$ minimizes the total path lengths of $N$ robots by shortening the distance to their target points, $f_2$ prevents robot-to-robot collisions, and $f_3$ avoids surrounding obstacles.

The mathematical expression of $f_1$ is as follows [47]:

$$f_1 = \sum_{i=1}^{N} \sqrt{\left(x_i^n - x_i^T\right)^2 + \left(y_i^n - y_i^T\right)^2} \tag{4}$$

where $(x_i^n, y_i^n)$ with $i = 1, \ldots, N$ indicates the next positions of each robot produced by the algorithm, and $(x_{ri}^T, y_{ri}^T)$ are the corresponding target points.

The mathematical formulation of $f_2$ is as follows [50]:

$$f_2 = \begin{cases} \sum_{j=1}^{N} \sum_{k=1}^{N} 1/D(j,k), & \text{if } D(j,k) > Q \\ P, & \text{otherwise} \end{cases}, \tag{5}$$

in which $D(j,k)$ indicates the distance between the $j$-th and $k$-th $(j \neq k)$ robots, and $P$ is the penalty term, which should be set to a very large value. Let $P = 10000$ here. And $Q$ stands for the safety distance, represented as $Q = 2 \times L$, with the diagonal length $L$.

When the robot senses its surroundings, it simulates radar detection to scan the area in front. As shown in Fig. 3, the robot detects obstacles in $M$ directions.

The mathematical statement of $f_3$ is as follows [51]:

$$f_3 = \begin{cases} \min \sum_{j=1}^{N} (1/D(j,dir)), & \text{if all } D(j,dir) > Q \\ P, & \text{otherwise} \end{cases}, \tag{6}$$

where $D(j,dir)$ indicates the maximum distance between the $j$-th robot $(j = 1, \ldots, N)$ and all obstacles when traversing all $dir$ $(dir = 1, \ldots, M)$ detection directions, and the penalty term $P$ should be set to a large value. If there is an obstacle between the current and next positions of the robot and condition $D(j,dir) > Q$ is satisfied, then $P$ remains at 10,000.

## 3. Basic knowledge

### 3.1. MOOP overview

The MOOP with multiple competing objectives can be expressed as [52]:

$$\text{Objective function}: \quad F(X) = \min\{f_1(X), f_2(X), \ldots, f_G(X)\}, \tag{7}$$

$$\text{Constraints}: \quad R_i^{\text{lower}} \leq x_i \leq R_i^{\text{upper}}, \ x_i \in X, \tag{8}$$

where $f_g(X)$ is the vector-valued objective function for $g = 1,2,\ldots,G$, and $R_i^{\text{lower}}$ and $R_i^{\text{upper}}$ are the boundaries of the $i$-th variable $x_i$ for $i = 1,2,\ldots, d$.

*Pareto dominance:* For two candidate solutions $Y = (y_1, y_2, \ldots, y_d)$ and $Z = (z_1, z_2, \ldots, z_d)$, $Y$ Pareto dominates $Z$ (indicated as $Y \prec Z$) if and only if:

**Condition 1.** Forall $g \in \{1, \ldots, G\}, f_g(Y) \leq f_g(Z)$;

**Condition 2.** There exists at least one $g \in \{1, \ldots, G\}$ such that $f_g(Y) < f_g(Z)$.

*Pareto front(PF)*: Given a finite solution set $P = \{X_1, X_2 \ldots X_n\}$, the pareto front is defined as:

$$PF = \{X \in P | \neg \exists Y \in P \text{ s.t. } Y \prec X\}, \tag{9}$$

where $X$ satisfies $\forall x_a, x_b \in X, x_a \neq x_b \Rightarrow \neg(x_a \prec x_b)$.

*Non-dominated sorting(NDS)*: Given a finite solution set $P = \{X_1, X_2 \ldots X_n\}$, a non-dominated sorting divides it into several non-dominated fronts through Eq. (9). Specifically, the first non-dominate $PF_1$ is identified from $P$ through Eq.(9), Subsequently, the second front $PF_2$ is extracted from the remaining solutions $P \backslash PF_1$, and the process continues iteratively until all solutions are assigned to their corresponding fronts.

### 3.2. Standard MOAHA

MOAHA begins by initializing the population and visit table. The population initialization formula is:

$$x_i = low + r \cdot (up - low) \quad i = 1, \ldots, n, \tag{10}$$

where $x_i$ denotes the position of the $i$ th food source, $n$ represents the population size, $low$ and $up$ represent the upper and lower bounds in the $d$-dimensional problem, respectively $r \in [0,1]$.

The visit table for food source is initialized as follows:

$$vt_{i,j} = \begin{cases} 0 & \text{if } i \neq j \\ null & i = j \end{cases} \quad i = 1, \ldots, n; j = 1, \ldots, n, \tag{11}$$

where $vt_{i,j} = 0$ (for $i \neq j$) indicates that in the current iteration, the $i$-th hummingbird has just visited the food source of the $j$-th hummingbird.

Following initialization, a flight skill is randomly selected. Hummingbirds are assumed to have three distinct flight skills, which are described as follows:

*Axial flight:* Through axial flight, hummingbirds can traverse any

**Algorithm 1**
Pseudo-code for MOAHA.

```
Input: n, dim, Maxiter, low, up, Max_Archive;
Output: Archive;
Initialize the population p using Eq.(10);
Initialize the visit table using Eq.(11);
Initialize archive with non-dominated solutions;
while t ⩽ Maxiter do
    for i=1: pop do
        D(i) ← Randomly perform Eq.(12), Eq(13), Eq.(14);
        if rand ⩽ 0.5 then // Guided foraging
            Generate a new candidate solution v_i with Eq.(15);
            Update solution with Eq.(18);
            Update visit table for each source;
        else // Territorial foraging
            Generate a new candidate solution v_i with Eq.(16);
            Update solution with Eq.(18);
            Update visit table for each source;
        end
    end
    if mod (t, 2n) ==0 then // Migration foraging
        Generate a new candidate solution v_i with Eq.(17);
        Update solution with Eq.(18);
        Update visit table for each source;
    end
    Perform NDS on population;
    DECD controls external archive size;
end
```

coordinate axis in the search space. The mathematical definition of this behavior is as follows:

$$D_{Af}{}^{(i)} = \begin{cases} 1, if\ i = randi([1,d]) \\ 0, else \end{cases}, \tag{12}$$

where $d$ is the dimension of the solution vector.

*Diagonal flight*: Diagonal flight can move between opposite corners of a rectangular search space. The mathematical definition of this behavior is as follows:

$$D_{Df}{}^{(i)} = \begin{cases} 1, if\ i = P(j), j \in [1,k], \\ k \in [2, [r_1 \cdot (d-2)] + 1] \\ P = randperm(k) \\ 0, else \end{cases}, \tag{13}$$

where $p$ is a random permutation from 1 to $k$ without duplicates, $r_1$ indicates a uniformly distributed random variable in the interval of [0,1].

*Omnidirectional flight*: Omnidirectional flight enables hummingbirds to travel in the direction of each axis projected onto the search space. This flight mode is represented as follows:

$$D_{Of}{}^{(i)} = 1, \quad i = 1, \ldots, d. \tag{14}$$

The next step after selecting a flight skill is to randomly choose a foraging strategy from either guided foraging or territorial foraging, which are described as follows.

*Guided foraging*: In MOAHA, during the guided foraging phase, each hummingbird selects the individual with the highest visitation level recorded in the visit table as its guiding target. The movement is formulated as:

$$v_i(t+1) = x_{i,gui}(t) + D \cdot a \cdot \left(x_i(t) - x_{i,gui}(t)\right), \tag{15}$$

where $v_i(t+1)$ is the newly generated solution for the $i$-th hummingbird at iteration $t+1$, $x_i(t)$ is the current position of the $i$-th hummingbird at iteration $t$, $x_{i,gui}(t)$ represents the guide individual for the $i$-th hummingbird, and $D$ is the flight skill randomly selected from Eqs.(12)-(14), $a$ is a guiding factor drawn from a standard normal distribution $N(0,1)$.

*Territorial foraging*: Hummingbirds forage within their territorial boundaries. This phase generates new food source solutions through local search, formula as follows:

$$\begin{cases} v_i(t+1) = x_i(t) + D \cdot a \cdot x_i(t)\ rand < 0.5 \\ v_i(t+1) = x_i(t) + D \cdot a \cdot x_a(t)\ rand \geq 0.5 \end{cases}, \tag{16}$$

where $x_a(t)$ is a solution randomly selected from the PF to augment the diversity of MOAHA solutions.

Moreover, in MOAHA, based on NDS, individuals in the least favorable front are selected for migration. The migratory foraging behavior is formulated as follows:

$$x_{wor}(t+1) = low + r \cdot (up - low),\ wor \in F_{end}, \tag{17}$$

where $F_{end}$ represents the worst front.

The next step after chooses a foraging strategy is to update the solution and visit table. Updates to the solution in MOAHA are based on NDS, the solution update formula is as follows:

$$x_i(t+1) = \begin{cases} v_i(t+1), v_i \in F_p,\ x_i \in F_q\ and\ p < q \\ \begin{cases} v_i(t+1), v_i \in F_p,\ x_i \in F_q, p = q\ and\ rand < 0.5 \\ x_i(t), v_i \in F_p,\ x_i \in F_q, p = q\ and\ rand \geq 0.5 \end{cases} \\ x_i(t), v_i \in F_p,\ x_i \in F_q\ and\ p > q, \end{cases} \tag{18}$$

where $F_p$ and $F_q$ are the $p$-th and $q$-th fronts obtained from NDS, respectively, $p < q$ represents $F_p$ dominating $F_q$, $p = q$ represents $F_p$ not

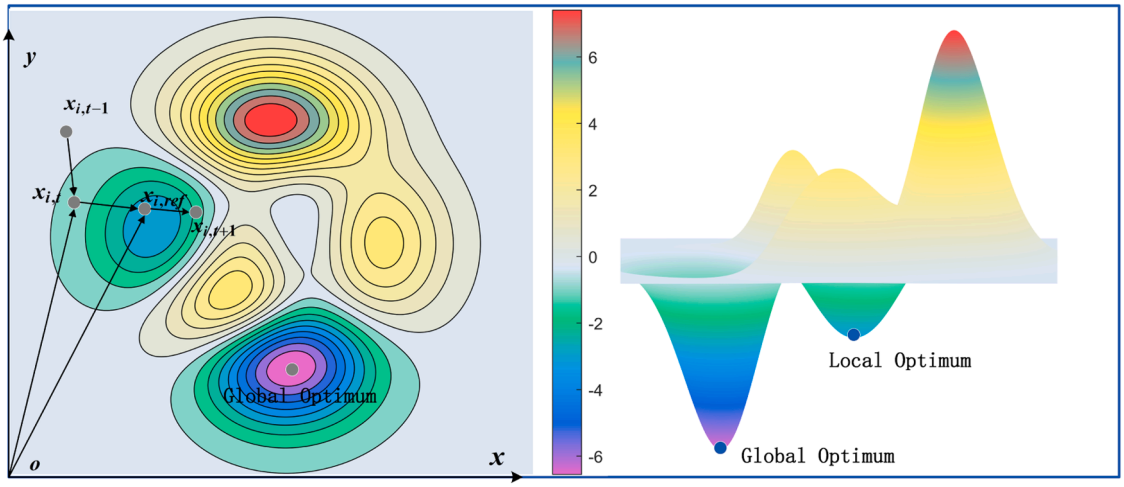



**Fig. 4.** Local optimum trap.

dominating $F_q$, $p > q$ represents $F_q$ dominating $F_p$. Additionally, the update rule for visit table is as follows. The visit level of individual $i$ to all other individuals increased by 1. If $p < q$, all other individuals set their visitation level to individual $i$ to the maximum value.

For clarity, Algorithm 1 provides the pseudocode for the basic MOAHA.

### 3.3. Reinforcement learning

Reinforcement learning is a machine learning paradigm in which an agent learns to maximize cumulative reward by interacting with an environment [53]. At each iteration $t$, the agent selects an action $a_t$ based on the current state $s_t$, leading the environment to transition to a next state $s_{t+1}$ and return a reward $r_t$.

As a reinforcement learning method, Q-learning estimates the action-value function via a Q-table [54], with each entry storing the expected return of taking a specific action in a given state. However, Q-tables are limited to discrete state spaces. In contrast, DQN employ deep neural networks to approximate the action-value function over continuous state spaces [55], where the network receives a state as input and outputs the corresponding Q-values for all possible actions. DQN stores collected experience tuples in a replay buffer to form the training dataset, and alternates between action selection and network updates. The network is optimized via gradient descent by minimizing the following loss function:

$$loss = \frac{1}{|T|}\sum_{t\in T}\left(r_t + \gamma \max_{a'\in A} Q(s_{t+1}, a') - Q(s_t, a_t)\right)^2, \tag{19}$$

where $T$ represents the training set, $r_t$ represents current reward, $A$ represents a set of actions, $\gamma$ represents the discount factor, $\max_{a'\in A} Q(s_{t+1}, a')$ represents max output of the target network with input vector $s_{t+1}$, $Q(s_t, a_t)$ represents the $t$-th output neuron of the online network with input vector $s_t$.

## 4. Proposed SFA-MOAHA

### 4.1. Motivations

MRPP encounters significant challenges in real-world environments, including high-dimensional search spaces, complex obstacle layouts, inter-robot collision risks, and the need to balance multiple objectives. Therefore, path planning algorithms must accurately generate feasible paths while avoiding convergence to local optima. Although the MOAHA performs well on standard optimization problems, its direct application to MRPP reveals notable limitations. First, the three flight skills in MOAHA, namely axial flight, diagonal flight, and omnidirectional flight, each use a fixed parameter value of 1. This limits the flexibility of updating each dimension during solution generation and consequently reduces the algorithm's global exploration capability in complex environments. Second, both the guided foraging (Eq. (15)) and territorial foraging (Eq. (16)) strategies rely solely on individuals within the population for guidance. This often leads to clustering of individuals, loss of population diversity, premature convergence, and local optima. Furthermore, MOAHA lacks an adaptive mechanism for selecting flight skills and foraging strategies based on real-time individual and population states, making it difficult to effectively balance exploration and exploitation during the search process. Therefore, enhancing exploration and exploitation capabilities of MOAHA to achieve this balance can effectively address the challenges of the MRPP.

#### 4.1.1. Adaptive spiral local search in omnidirectional flight

The basic MOAHA has relatively low level of exploitation ability. To augment the exploitation ability of MOAHA, the omnidirectional flight skill is augmented with a spiral approximation technique [56] is incorporated into the omnidirectional flight skill. In this strategy, the hummingbird spirals towards the target in each dimension. Therefore, Eq. (14) can be rewritten as:

$$D_{of}^{(i)} = e^{b\times r} \times \sin(2\pi r), \quad i = 1, \ldots, d, \tag{20}$$

where $r$ is an evenly distributed random variable in the interval of $[-1,1]$, and $b$ represents the decay index adjusted to enhance flexibility across different search stages. Specifically, $b$ is an adaptive variable influenced by the evaluated function and the local optimum index, expressed as follows:

$$b = L + 1 - FES/FES_{\max}, \tag{21}$$

where $FES$ represents the number of objective function evaluations, $FES_{\max}$ is the maximum number of evaluations, and $L$ stands for the number of consecutive iterations during which the current individual position remains unchanged.

When omnidirectional flight is activated, it updates all dimensions. Eq. (20) uses a composite exponential-sine function to dynamically adjust the step size for each dimension, enabling the adaptive spiral local search in omnidirectional flight to effectively explore the target region. Notably, when an individual becomes trapped in a local optimum, it adjusts its flight stride via $L$ to enhance its exploratory ability. This step-size adjustment mechanism allows the adaptive spiral local search in omnidirectional flight to improve convergence accuracy while helping

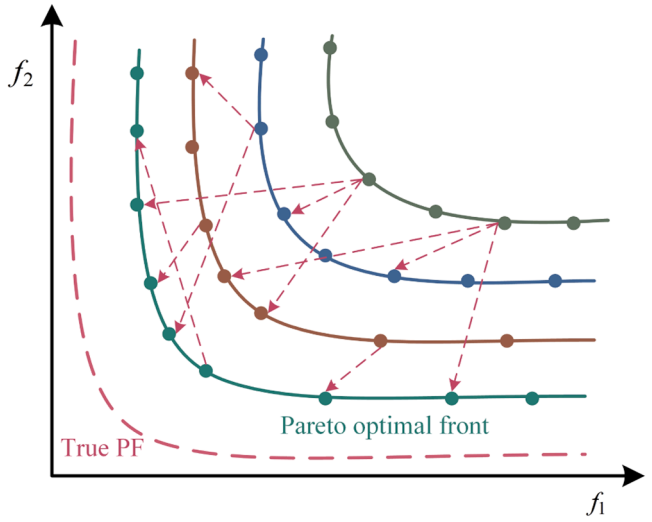



**Fig. 5.** The PF distribution under the forward-fusion learning strategy.

MOAHA avoid local optima.

*4.1.2. Proposed a forward fusion learning foraging based on PF dominance levels*

In the guided foraging phase of MOAHA, hummingbirds are drawn to the most-visited food source. During territorial foraging, they search within their territories. During the migration foraging phase, new individuals will be randomly generated from underperforming individuals. However, these strategies have flaws. When approaching a local optimum, a hummingbird may get trapped, unable to fully exploit available information and causing the local optimization issue. As displayed in Fig. 4, the points in the purple area indicate the theoretical global optimum, while the green and yellow areas indicate local optima. Variable $x_{i,t-1}$, $x_i$, $x_{i,ref}$ and $x_{i,t+1}$ represent the previous, current, reference position, and next-moment position of a hummingbird respectively. If the reference position is a local optimum, vector calculations show that hummingbirds can easily fall into a local optimum trap.

MOAHA divides populations into PFs of different dominance levels

**Algorithm 2**
Train/update network.

1: **Input:** *ep* (experience replay), *bs* (required size of training data);
2: **Output:** DQN;
3: initialize network weight *w*;
4: $T$←randomly sample *bs* records as the training data;
5 Normalize all items of $T$;
5: $S_t \leftarrow \{t_1, t_2, t_3, t_4, t_5\}, t \in T$; //state
6: $a_t \leftarrow \{t_6\}, t \in T$;//action
7: $r_t \leftarrow \{t_7\}, t \in T$;//reward
8: $S_{t+1} \leftarrow \{t_8, t_9, t_{10}, t_{11}, t_{12}\}, t \in T$; //next state
9: DQN←train the DQN network weights using Eq.(19) as loss function

based on NDS. Countering the tendency of hummingbirds to stagnate in local optima, this study devises a forward fusion learning approach grounded in PF dominance hierarchies. The position update is governed by:

$$v_i(t+1) = x_i(t) + D \times a \times \sum_{j=1}^{n} \left(x_j(t) - x_i(t)\right), \tag{22}$$

where $n$ is the number of PFs in the population whose dominance level is higher than that of $x_i(t)$ in the PF, $j = 1...n$, $x_j(t)$ is an individual randomly selected from the PFs. Fig. 5 depicts the forward fusion learning strategy based on the PF dominance hierarchy. The red dashed line stands for the true PF, and the other colored lines represent PFs with different dominance levels. The red dotted arrow indicates that individuals randomly select a solution from the PFs with high dominance levels as the learning target. Therefore, forward fusion learning strategy based on PF dominance levels can effectively utilize population information to enhance exploration capabilities of MOAHA.

*4.1.3. DQN for search strategy selection*

While MOAHA incorporates multiple search strategies, it lacks an intrinsic mechanism to dynamically select the most suitable operator under the varying conditions typical of MRPP. To overcome this limitation, we leverage the adaptive learning capability of DQN, which enables agents to learn optimal policies through interaction with the environment and reward feedback. As depicted in Fig. 6, the DQN framework comprises two neural networks: an online network and a

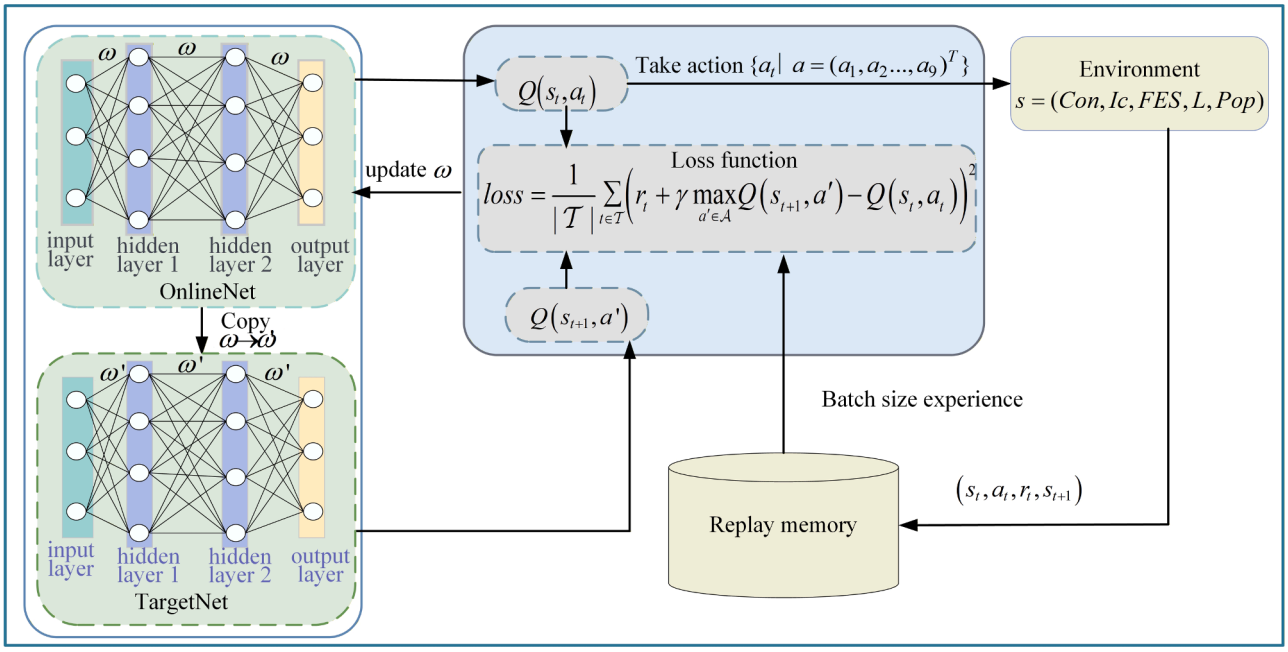



**Fig. 6.** Adaptive selection of search strategy based on DQN.

**Algorithm 3**
Select action.

1: **Input:** $\varepsilon$ (possibility of greedy), $s_t$ (current population state), $A$ (operator set), DQN;
2: **Output** $a_t$ (the selected operator);
4: $p \leftarrow$ generate a random number in [0,1];
5: **if** $p \le \varepsilon$ **then**
6: $s \leftarrow$ normalize all items of the state data s;
7: $i = \text{argmax}\, Q(s,a)$ ;
8: **else**
9: $i \leftarrow$ generate a random number in $\{1,2...9\}$ ;
10: **end**
11: $a_t \leftarrow$ the $i$-th operator in $A$ from Eq.(27);

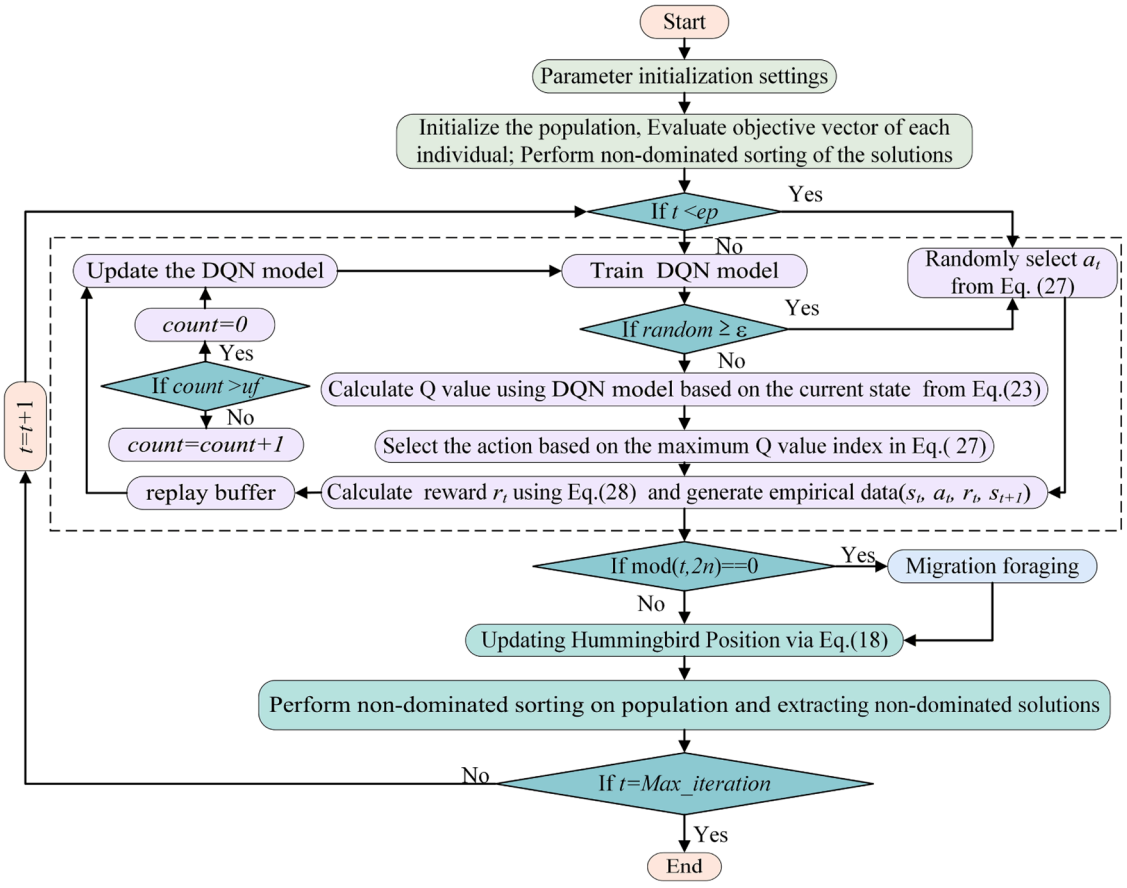



**Fig. 7.** Flowchart of SFA-MOAHA.

target network. The online network selects an action $a_t$ based on the current state $s_t$ to interact with the environment, generating experience data $(s_t, a_t, r_t, s_{t+1})$. This data is stored in a replay buffer and used to train the online network. In our method, individuals select search strategies via the online network before updating their positions. Subsequently, MOAHA trains the online network by feeding back solutions and corresponding fitness improvements as state and reward signals. This strategy selection creates an adaptive learning mechanism that balances exploration and exploitation in response to real-time population feedback. It prioritizes global search in uncertain areas for high-quality solutions and local optimization in stable areas to refine existing ones. Through such adjustment, MOAHA can better balance exploration and exploitation, enhancing its performance.

When using DQN to adaptively select the search strategy of the MOAHA, the key lies in designing appropriate state, action and reward mechanisms.

① State: During SFA-MOAHA iterations, the agent's state is determined by factors like convergence, diversity, objective function evaluations, local optimal index, and position. A state is defined by using five components, and the state set is as follows:

$$S = \{s \mid s = (Con, Ic, FES, L, Pop)\}, \tag{23}$$

where *FES* are the number of target function assessments, *Pop* indicates

**Algorithm 4**
Pseudo-code for SFA-MOAHA.

```
Input: n, dim, Maxiter, low, up, Max_Archive;
Output: Archive;
Initialize the population p using Eq.(10);
Initialize the visit table using Eq.(11)
Initialize DQN parameters: learning rate, buffer size, batch size;
replay buffer ← Create replay buffer with capacity buffer size;
Initialize variables: count=0, model built=0, uf(update frequency of DQN),
while t ≤ Maxiter do
    for i=1: pop do
        s_t ← Compute current state with Eq. (21);
        if Size of the replay buffer < buffer size
            a_t ← Randomly select an action from Eq.(27);
        else
            [online network, target network] ←Algorithm 2 train DQN using replay buffer;
            model_built=1;
            a_t ←Select action through algorithm 3;
        end
        Generate a new candidate solution v_i through a_t;
        Update solution with Eq.(18);
        Update visit table for each source;
        s_{t+1}← Compute next state with Eq.(23);
        r_t ← Compute reward with Eq.(28)
        replay buffer ← (s_t, a_t, r_t, s_{t+1});
    end
    if model_built then
        count ← count + 1;
        if count>uf then
            [online network, target network] ←Algorithm 2 update DQN using replay buffer;
            Reset count=0;
        end
    end
    if mod (t, 2n) ==0 then // Migration foraging
        Generate a new candidate solution v_i with Eq.(17);
        Update solution with Eq.(18);
        Update visit table for each source;
    end
    Perform NDS on population;
    DECD controls external archive size
end
```

**Table 1**
Parameter specifications of the competing methods.

| Algorithms | Parameters | Value |
|---|---|---|
| MOPSO | number of grids | 10 |
| | inertia weight | 0.4 |
| NSGA-II | crossover percentage | 0.7 |
| | mutation percentage | 0.4 |
| | mutation rate | 0.02 |
| | number of mutants | 40 |
| MOGWO | number of grids | 10 |
| | leader selection pressure | 4 |
| | repository member selection pressure | 2 |
| NSWOA | parameter less | - |
| LERD | parameter less | - |
| MOEADDQN | parameter less | - |
| MOAHA | parameter less | - |
| SFA-MOAHA | parameter less | - |

the current position of individual, and $L$ is the number of consecutive iterations during which the position of individual remains unchanged, $Con, Ic$ represent the population states, and $FES$, $L$, $Pop$ represents the individual state.

Convergence is measured by the average sum of objective function values, which shows how close it is to the true PF $Con$, and is expressed as:

$$Con = \frac{\sum_{j=1}^{m} f_j(x)}{N}, \tag{24}$$

where $f_j(x)$ is the value of the $j$-th objective function and $N$ denotes the population size. As the population gets closer to the TPF, the value of $Con$ decreases. The population diversity metric is measured utilizing the formula of inertia moments, which is defined as:

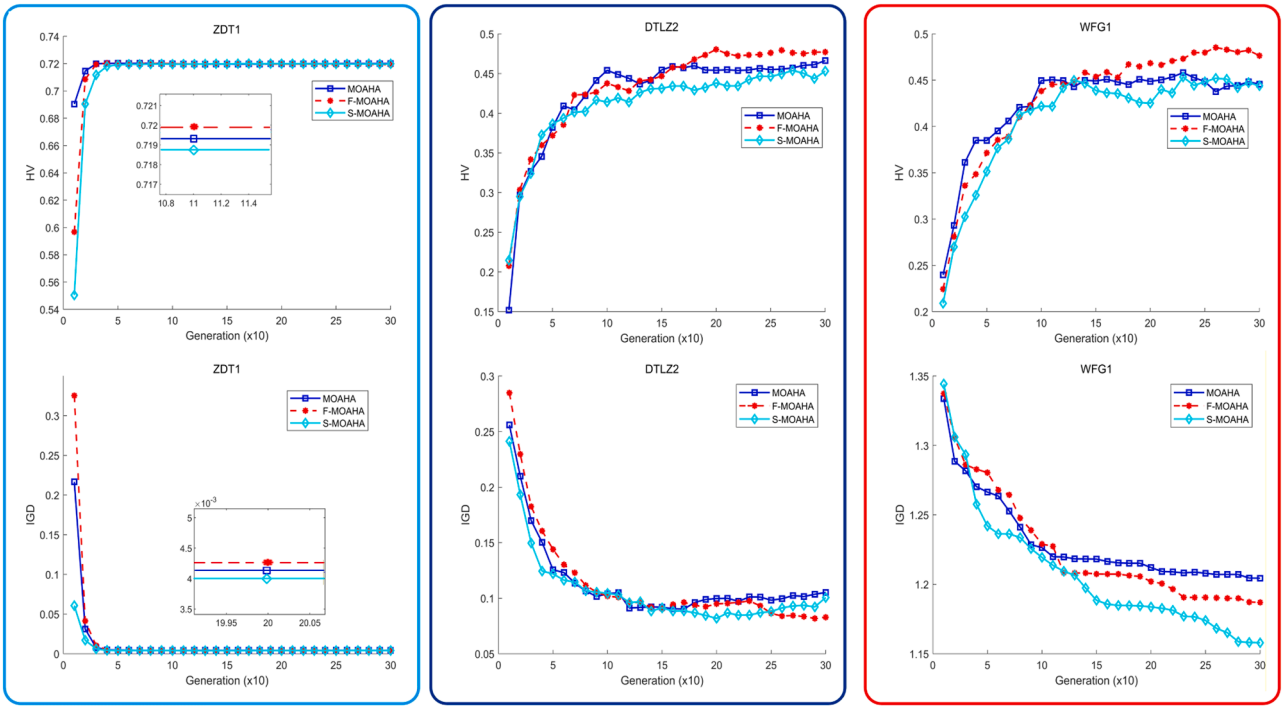



**Fig. 8.** Comparison of MOAHA, F-MOAHA, and S-MOAHA performance on ZDT1, DTLZ2, and WFG1.

$$I_C = \sqrt{\sum_{i=1}^{N}\sum_{D=1}^{D}(x_{id} - c_d)^2}, \tag{25}$$

where $x_{id}$ denotes the $d$-dimensional coordinate of the $i$-th search agent at iteration $t$, $N$ and $D$ representing population size and dimensionality respectively, and during each iteration, $c_d$ measures the dispersion within the population relative to its mass center, which is computed as:

$$c_d = \sum_{i=1}^{N} x_{id} / D. \tag{26}$$

The choice of search strategy relies on a synthesized evaluation of the current population state and individual performance indicators. To improve network training efficiency, all state values are normalized before being fed into the network.

② Action: The action set consists of nine search strategies, formed by combining three flight strategies and three foraging strategies. The three flight strategies include axial flight (Eq. (12)), diagonal flight (Eq. (13)), and adaptive spiral local search in omnidirectional flight (Eq. (20)). The three foraging strategies comprise guided foraging (Eq. (15)), territorial foraging (Eq. (16)), and forward fusion learning foraging based on PF dominance levels (Eq. (22)). Thus, the agent's action $a_t$ can be defined as:

$$a_t \in A, A = (a_1, a_2 \ldots, a_9)^T, i = 1, 2, \ldots, 9. \tag{27}$$

③ Reward: The reward mechanism is the key to optimize policy parameters in reinforcement learning. The reward rule $r$ is based on the performance comparison between the new individual and the current one, and it can be expressed as follows:

$$r = \begin{cases} 1 \ newposition \prec currentposition \\ -1 \ newposition \succ currentposition \\ 0 \ otherwise \end{cases}, \tag{28}$$

Based on the above definitions and proposals, a deep Q-learning model is developed for selecting search strategy in SFA- MOAHA. It consists of four main procedures:

(a) Foraging: Each individual acts as an agent. SFA-MOAHA employs a DQN to choose a search strategy and generate a new position. A mechanism-*greedy* balances exploration and exploitation: the agent randomly selects an action with probability $\varepsilon$ for exploration and the action with the max Q-value estimated by DQN with probability $1 - \varepsilon$.

(b) Interaction: After strategy selection, the agent interacts with the environment, which provides feedback based on the foraging behavior.

(c) Learning: After receiving feedback from the environment, the agent refines its strategy through learning. It stores the current iteration record in the experience replay buffer and uses sampled data to train the DQN. The current state is used as input and the Q-value of the policy is output. The pseudocode for train network is provided in Algorithm 2.

(d) Decision-making: After training, the DQN enables the agent to make decisions based on the current state. It estimates Q-values for all actions and chooses the one with the highest Q-value as the strategy, replacing the random selection strategy in steps 3 and 4 of MOAHA. The pseudocode for select action is provided in Algorithm 3.

Based on the preceding analysis, the three strategies proposed in this paper are integrated with MOAHA to form the SFA-MOAHA framework. The detailed implementation process of SFA-MOAHA is outlined as follows.

**Step 1:** Initialize the parameters: population size $n$, variable dimension *dim*, the range of design variables ($low$, $up$), maximum number of iterations *Max Iteration* and DQN parameters.

**Step 2:** Initialize the population through Eq. (10); Evaluate the objective vector of each solution; Initialize the visit table through Eq. (11); Perform NDS on the population according to Eq.(9).

**Step 3:** Select search strategy: When current iteration $t <$ *Max Iteration*, each hummingbird either randomly selects a search strategy from Eq. (27) with probability $\varepsilon$; Through Algorithm 3 to choose the optimal strategies based on the current state Eq. (23);

**Table 2**
IGD values obtained by all comparison algorithms.

| Problem | MOAHA | MOGWO | NSWOA | MOPSO | NSGA-II | MOEADDQN | LERD | SFA-MOAHA |
|---|---|---|---|---|---|---|---|---|
| ZDT1 | 4.0680e-3 | 1.2504e-2 | 5.4331e-3 | 5.9362e-3 | 2.8148e-2 | 3.2388e-2 | 3.9265e-3 | **3.9105e-3** |
| | (9.37e-5)- | (3.34e-3)- | (3.17e-4)- | (2.82e-4)- | (2.34e-3)- | (1.26e-1)- | (2.38e-5)- | **(9.69e-5)** |
| ZDT2 | 4.0972e-3 | 5.0878e-2 | 4.6814e-3 | 5.7565e-3 | 3.6573e-2 | **3.8274e-3** | 3.9617e-3 | 3.9699e-3 |
| | (8.95e-5)- | (1.52e-1)- | (1.82e-4)- | (2.06e-4)- | (3.22e-3)- | **(2.41e-5)**+ | (1.83e-5)+ | (5.86e-5) |
| ZDT3 | 4.4582e-3 | 1.1548e-2 | 5.1596e-3 | 5.3028e-3 | 2.5270e-2 | 1.0954e-2 | 6.9941e-3 | **4.4454e-3** |
| | (8.73e-5)= | (2.26e-3)- | (1.93e-4)- | (1.86e-4)- | (2.63e-3)- | (4.31e-5)- | (4.44e-5)- | **(5.14e-5)** |
| ZDT4 | 3.9035e-3 | 1.4941e-1 | 4.9722e-3 | 2.3138e-1 | 8.2201e-1 | 1.5262e-2 | 4.1130e-3 | **3.8302e-3** |
| | (4.90e-5)- | (3.14e-1)- | (2.76e-4)- | (3.34e-1)- | (8.32e-2)- | (1.98e-2)- | (6.61e-5)- | **(2.99e-5)** |
| ZDT6 | 3.1028e-3 | 8.1556e-3 | 3.7198e-3 | 7.2874e-3 | 2.3550e+0 | 3.1322e-3 | **3.0428e-3** | 3.1460e-3 |
| | (3.61e-5)+ | (2.25e-3)- | (1.99e-4)- | (1.89e-3)- | (7.71e-2)- | (1.19e-5)= | **(1.94e-5)**+ | (6.91e-5) |
| DTLZ1 | 1.0640e+0 | 9.5238e+0 | 2.4837e+0 | 1.6233e+1 | 1.1595e+0 | **2.1680e-2** | 1.1350e-1 | 4.4573e-1 |
| | (6.59e-1)- | (4.35e+0)- | (1.65e+0)- | (3.32e+0)- | (5.56e-1)- | **(1.00e-3)**+ | (2.24e-1)+ | (2.91e-1) |
| DTLZ2 | 9.6197e-2 | 3.7890e-1 | 8.2569e-2 | 1.0322e-1 | 6.9504e-2 | **5.5916e-2** | 6.5243e-2 | 7.6463e-2 |
| | (4.80e-3)- | (4.50e-2)- | (3.52e-3)- | (5.61e-3)- | (4.35e-3)+ | **(5.95e-4)**+ | (1.52e-3)+ | (5.19e-3) |
| DTLZ3 | 3.0445e+1 | 1.7317e+2 | 2.5825e+1 | 1.8573e+2 | 1.5243e+1 | **1.1840e+0** | 4.6295e+0 | 1.3504e+1 |
| | (1.83e+1)- | (4.21e+1)- | (2.19e+1)- | (1.38e+1)- | (4.88e+0)= | **(2.36e+0)**+ | (6.45e+0)+ | (6.76e+0) |
| DTLZ4 | 1.4827e-1 | 1.3284e-1 | 2.0403e-1 | 1.5016e-1 | **8.2477e-2** | 1.0821e-1 | 1.4716e-1 | 9.7729e-2 |
| | (2.21e-2)- | (9.27e-2)= | (2.86e-2)- | (4.94e-2)- | **(1.59e-2)**+ | (1.97e-1)- | (1.45e-1)- | (2.20e-2) |
| DTLZ5 | 1.0646e-2 | 1.5269e-2 | 1.0448e-2 | 2.6963e-2 | 5.6360e-3 | 2.2825e-2 | 2.3892e-2 | **5.1303e-3** |
| | (1.23e-3)- | (6.21e-3)- | (1.14e-3)- | (3.28e-3)- | (3.43e-4)- | (6.77e-5)- | (4.05e-4)- | **(8.92e-4)** |
| DTLZ6 | 4.4392e-3 | 1.0089e-2 | 5.1227e-3 | 6.9029e-3 | 7.0954e+0 | 2.2881e-2 | 2.3974e-2 | **4.4335e-3** |
| | (9.15e-5)- | (1.79e-3)- | (1.66e-4)- | (1.39e-3)- | (2.03e-1)- | (1.71e-5)- | (3.13e-4)- | **(8.33e-5)** |
| DTLZ7 | **2.515e-2** | 4.6182e-1 | 8.3039e-2 | 1.3521e-1 | 2.8347e-2 | 1.5430e-1 | 7.8079e-1 | 2.5191e-2 |
| | **(1.09e-3)**= | (1.24e-2)- | (2.36e-3)- | (5.17e-3)- | (2.51e-3)- | (3.54e-3)- | (1.00e-1)- | (7.59e-4) |
| WFG1 | 1.1913e+0 | 1.2734e+0 | 1.1767e+0 | 1.2236e+0 | 1.5662e+0 | 5.5809e-2 | **2.1224e-2** | 9.2692e-1 |
| | (1.35e-2)- | (1.45e-2)- | (1.02e-2)- | (1.37e-2)- | (6.18e-2)- | (1.67e-1)+ | **(3.96e-3)**+ | (6.87e-2) |
| WFG2 | 2.5093e-2 | 3.6661e-2 | 6.7354e-2 | 1.9446e-2 | 1.8409e-1 | 2.0536e-2 | 1.9438e-2 | **1.8044e-2** |
| | (4.12e-3)- | (1.30e-2)- | (2.08e-2)- | (2.16e-3)- | (6.50e-2)- | (9.98e-5)- | (6.32e-4)- | **(9.98e-3)** |
| WFG3 | 2.0734e-2 | 4.5062e-2 | 2.3998e-2 | 2.1493e-2 | 1.6122e-2 | 1.3319e-2 | **1.2450e-2** | 1.6941e-2 |
| | (1.31e-3)- | (9.10e-3)- | (3.14e-3)- | (1.11e-3)- | (1.02e-3)+ | (4.68e-5)+ | **(3.38e-4)**+ | (8.15e-4) |
| WFG4 | 6.9865e-2 | 8.5249e-2 | 7.1567e-2 | 7.2704e-2 | 7.4229e-2 | **1.4241e-2** | 1.8051e-2 | 6.9121e-2 |
| | (8.33e-4)- | (4.05e-3)- | (1.64e-3)- | (1.12e-3)- | (5.33e-3)- | **(8.06e-4)**+ | (2.46e-3)+ | (1.37e-3) |
| WFG5 | 6.9443e-2 | 8.3149e-2 | 7.1163e-2 | 7.2724e-2 | 7.4666e-2 | 6.9374e-2 | 6.9482e-2 | **6.9075e-2** |
| | (1.59e-3)= | (4.35e-3)- | (1.64e-3)- | (1.23e-3)- | (4.65e-3)- | (3.46e-3)- | (3.83e-3)- | **(1.69e-4)** |
| WFG6 | 2.2501e-2 | 3.1873e-2 | 2.9161e-2 | 2.0083e-2 | 2.5213e-2 | 1.8520e-2 | 1.8235e-2 | **1.7420e-2** |
| | (1.84e-3)- | (7.56e-3)- | (5.55e-3)- | (1.01e-3)- | (1.32e-2)- | (2.48e-4)- | (1.42e-4)- | **(1.11e-3)** |
| WFG7 | 2.2663e-2 | 2.8656e-2 | 2.6310e-2 | 1.9846e-2 | 1.7366e-2 | 1.3245e-2 | **1.3157e-2** | 1.7946e-2 |
| | (1.98e-3)- | (4.27e-3)- | (2.49e-3)- | (9.58e-4)- | (1.54e-3)+ | (6.23e-5)+ | **(2.33e-4)**+ | (1.19e-3) |
| WFG8 | **1.9417e-1** | 3.1001e-1 | 2.1646e-1 | 3.0788e-1 | 2.9987e-1 | 2.0950e-1 | 2.2186e-1 | 2.0832e-1 |
| | **(5.07e-2)**= | (2.45e-2)- | (3.45e-2)= | (1.81e-2)- | (2.42e-2)- | (8.61e-2)- | (7.46e-2)- | (6.42e-2) |
| WFG9 | 2.4365e-2 | 3.7863e-2 | 2.6365e-2 | 2.5297e-2 | 2.4203e-2 | 4.4159e-2 | 2.3408e-2 | **2.3200e-2** |
| | (6.66e-4)- | (5.08e-3)- | (1.57e-3)- | (8.37e-4)- | (2.22e-3)- | (1.10e-1)- | (2.95e-3)- | **(6.83e-4)** |
| ±/= | 1/16/4 | 0/20/1 | 0/20/1 | 0/21/0 | 4/16/1 | 8/12/1 | 9/13/0 | - |

Evaluate the objective values of new candidate solutions, update the hummingbird positions by Eq. (18) and update updating the visit table; Store experience $(s_t, a_t, r_t, s_{t+1})$ in the replay buffer.

**Step 4:** Migratory foraging and position update: When $\mathrm{mod}(t, 2n) == 0$, update the hummingbird positions through migratory foraging using Eq. (17); Evaluate the objective values of new candidate solutions, update the positions through Eq. (18) and update the visit-table; Store experience $(s_t, a_t, r_t, s_{t+1})$ in the replay buffer.

**Step 5:** DQN Update: Periodically execute algorithm 2 to update the DQN weights using samples drawn from the replay buffer.

**Step 6:** Perform NDS on the population, and apply the DECD method to control the size of the external archive.

**Step 7:** Iteration judgment: After each iteration, increment the iteration number $t = t + 1$ if $t > Max\ Iteration$, output the external archive as the estimated PF; otherwise, return to **Step3**.

The flowchart of SFA-MOAHA is shown in Fig. 7 and the pseudo-code is given by algorithm 4.

### 4.2. The time complexity of the proposed SFA-MOAHA

The computational complexity of the SFA-MOAHA can be analyzed via its main components: population size $(N)$, number of objectives $(M)$, maximum iterations $(I)$, design variables $(D)$, DQN training times $(T)$, update times $(C)$ and number of neurons $(U)$.

The meta-heuristic initializes the total dimension of all individuals in the population before the iteration. So, the time complexity of the SFA-MOAHA initialization step is $O(N \times D)$. The complexity of non-dominated sorting is $O(\mathrm{I} \times M \times N^2)$.

The complexity of NDS process is $O(\mathrm{I} \times M \times N^2)$. The complexity of training DQN is $O(T \times U^2)$. The complexity of updating DQN is $O(C \times T \times U^2)$. The complexity of selecting a search strategy for DQN is $O(I \times U^2)$. Therefore, the overall complexity of the SFA-MOAHA can be represented as:

$$\begin{aligned} &O(SFA-MOAHA) = O(N \times D) + O(I \times M \times N^2) + O(T \times U^2) \\ &+O(C \times T \times U^2) + O(T \times U^2) \\ &\approx O(I \times M \times N^2 + C \times T \times U^2. \end{aligned} \tag{29}$$

## 5. Tests and experiments on numerical and engineering design problems

### 5.1. Experimental settings

The DQN within SFA-MOAHA employs the following architecture: an input layer of size $(d + 4)$, where $d$ represents the problem dimension, an output layer dimension matching the number of available search strategies, and two hidden layers comprising 128 and 64 neurons. Experience replay buffer capacity was fixed at 10,000 transitions. Network optimization utilized gradient descent (learning rate =0.001) with a

**Table 3**
HV values obtained by all comparison algorithms.

| Problem | MOAHA | MOGWO | NSWOA | MOPSO | NSGA-II | MOEADDQN | LERD | SFA-MOAHA |
|---|---|---|---|---|---|---|---|---|
| ZDT1 | 7.1967e-1 | 7.1062e-1 | 7.1748e-1 | 7.1668e-1 | 6.8537e-1 | 6.8751e-1 | 7.1894e-1 | **7.2014e-1** |
| | (2.28e-4)- | (3.08e-3)- | (5.00e-4)- | (3.88e-4)- | (2.89e-3)- | (1.43e-1)- | (1.10e-4)- | **(2.84e-4)** |
| ZDT2 | 4.4439e-1 | 4.1043e-1 | 4.4367e-1 | 4.4136e-1 | 3.9395e-1 | 4.4485e-1 | 4.4464e-1 | **4.4486e-1** |
| | (2.81e-4)- | (8.69e-2)- | (4.83e-4)- | (3.67e-4)- | (4.17e-3)- | (1.46e-4)= | (1.01e-4)- | **(2.41e-4)** |
| ZDT3 | 5.9985e-1 | 5.9773e-1 | 5.9930e-1 | 5.9941e-1 | 5.8441e-1 | 5.9793e-1 | 5.9897e-1 | **5.9991e-1** |
| | (6.60e-5)- | (3.72e-3)- | (1.59e-4)- | (8.98e-4)- | (2.12e-3)- | (5.87e-5)- | (1.13e-4)- | **(1.91e-4)** |
| ZDT4 | 7.2024e-1 | 6.0814e-1 | 7.1865e-1 | 5.0845e-1 | 2.8140e-2 | 7.0518e-1 | 7.1938e-1 | **7.2054e-1** |
| | (1.68e-4)- | (2.35e-1)- | (7.42e-4)- | (2.81e-1)- | (2.14e-2)- | (2.31e-2)- | (1.89e-4)- | **(5.81e-5)** |
| ZDT6 | 3.8884e-1 | 3.8223e-1 | 3.8817e-1 | 3.8484e-1 | 3.7795e-1 | 3.8883e-1 | **3.8891e-1** | 3.8879e-1 |
| | (5.35e-5)+ | (5.04e-3)- | (2.37e-4)- | (1.73e-3)- | (2.74e-4)- | (2.33e-4)- | **(2.07e-5)**+ | (7.43e-5) |
| DTLZ1 | 4.9148e-2 | 5.8561e-1 | 4.9208e-2 | 4.8251e-3 | 4.6549e-1 | **8.3655e-1** | 6.7995e-1 | 1.7492e-1 |
| | (1.77e-1)- | (8.23e-4)+ | (1.56e-1)- | (3.54e-2)- | (3.16e-1)+ | **(3.52e-3)**+ | (2.69e-1)+ | (2.83e-1) |
| DTLZ2 | 4.6122e-1 | 2.1455e-1 | 4.8386e-1 | 4.4757e-1 | 5.1549e-1 | **5.5621e-1** | 5.4154e-1 | 5.0773e-1 |
| | (1.28e-2)- | (2.66e-2)- | (7.90e-3)- | (1.03e-2)- | (9.44e-3)+ | **(9.44e-4)**+ | (2.13e-3)+ | (1.32e-2) |
| DTLZ3 | 1.6241e-1 | 4.2597e-3 | 4.5317e-5 | 5.0254e-3 | 3.2098e-1 | **3.3623e-1** | 2.1558e-1 | 2.0584e-1 |
| | (5.57e-1)- | (8.56e-3)- | (2.48e-4)- | (1.59e-2)- | (2.05e-1)+ | **(2.56e-1)**+ | (2.49e-1)+ | (7.25e-3) |
| DTLZ4 | 4.9255e-1 | 4.6378e-1 | 4.4586e-1 | 4.4322e-1 | 5.1083e-1 | 5.2390e-1 | 5.1595e-1 | **5.3229e-1** |
| | (1.37e-2)- | (5.77e-2)- | (1.53e-2)- | (2.66e-2)- | (1.71e-2)- | (1.04e-1)- | (6.32e-2)- | **(1.37e-2)** |
| DTLZ5 | 1.9345e-1 | 1.8953e-1 | 1.9276e-1 | 1.7556e-1 | 1.9863e-1 | 1.9046e-1 | 1.8927e-1 | **2.0127e-1** |
| | (1.04e-3)- | (7.05e-3)- | (1.16e-3)- | (5.11e-3)- | (4.73e-4)= | (2.81e-5)- | (3.82e-4)- | **(7.53e-4)** |
| DTLZ6 | 2.0028e-1 | 1.9262e-1 | 1.9987e-1 | 1.9887e-1 | 1.8946e-1 | 1.9047e-1 | 1.9041e-1 - | **2.0927e-1** |
| | (2.61e-5)= | (2.93e-3)- | (9.70e-5)- | (7.90e-4)- | (1.54e-4)- | (8.59e-6)- | (2.20e-4) | **(2.67e-5)** |
| DTLZ7 | **2.7251e-1** | 1.9941e-1 | 2.6296e-1 | 2.4034e-1 | 3.9738e-2 | 2.6012e-1 | 2.0240e-1 | 2.7102e-1 |
| | **(2.59e-3)**= | (3.96e-2)- | (1.77e-3)- | (4.57e-3)- | (8.65e-3)- | (6.70e-4)- | (8.26e-3)- | (5.73e-3) |
| WFG1 | 1.8875e-1 | 1.4741e-1 | 1.8014e-1 | 1.7236e-1 | 2.2339e-2 | 6.7391e-1 | **6.9155e-1** | 2.5923e-1 |
| | (3.32e-3)- | (3.93e-3)- | (3.97e-3)- | (3.48e-3)- | (2.04e-2)- | (8.46e-2)+ | **(1.93e-3)**+ | (2.08e-2) |
| WFG2 | 6.2828e-1 | 6.2209e-1 | 6.2014e-1 | 6.2867e-1 | 6.1400e-1 | 6.3189e-1 | **6.3237e-1** | 6.3202e-1 |
| | (1.03e-3)- | (3.10e-3)- | (2.91e-3)- | (6.10e-4)- | (6.08e-3)- | (1.72e-4)- | **(1.44e-4)**= | (8.66e-4) |
| WFG3 | 5.7480e-1 | 5.5998e-1 | 5.7113e-1 | 5.7485e-1 | 5.7776e-1 | **5.8137e-1** | 5.8083e-1 | 5.7744e-1 |
| | (8.39e-4)- | (4.25e-3)- | (2.01e-3)- | (6.84e-4)- | (9.58e-4)+ | **(1.19e-4)**+ | (3.56e-4)+ | (5.13e-4) |
| WFG4 | 3.0781e-1 | 2.9985e-1 | 3.0684e-1 | 3.0602e-1 | 3.0562e-1 | **3.4515e-1** | 3.4277e-1 | 3.0824e-1 |
| | (4.07e-4)= | (1.64e-3)- | (1.68e-3)- | (7.35e-4)- | (2.99e-3)- | **(8.66e-4)**+ | (1.37e-3)+ | (3.73e-4) |
| WFG5 | 3.0812e-1 | 3.0112e-1 | 3.0684e-1 | 3.0600e-1 | 3.0534e-1 | 3.0790e-1 | 3.0812e-1 | **3.0829e-1** |
| | (9.70e-4)= | (1.62e-3)- | (1.00e-3)- | (8.23e-4)- | (2.61e-3)- | (1.92e-3)+ | (2.21e-3)- | **(1.05e-4)** |
| WFG6 | 3.4124e-1 | 3.3602e-1 | 3.3536e-1 | 3.4217e-1 | 3.3605e-1 | 3.4390e-1 | 3.4267e-1 | **3.4394e-1** |
| | (7.14e-4)- | (4.60e-3)- | (3.45e-3)- | (5.16e-4)- | (8.59e-3)- | (3.22e-4)- | (1.59e-4)- | **(6.46e-4)** |
| WFG7 | 3.4180e-1 | 3.3974e-1 | 3.3912e-1 | 3.4266e-1 | 3.4177e-1 | 3.4336e-1 | 3.4280e-1 | **3.4402e-1** |
| | (6.61e-4)- | (1.80e-3)- | (1.45e-3)- | (4.83e-4)- | (1.41e-3)- | (1.11e-4)- | (2.43e-4)- | **(5.25e-4)** |
| WFG8 | 2.5698e-1 | 2.2507e-1 | 2.4770e-1 | 2.2415e-1 | 2.3441e-1 | **2.7075e-1** | 2.5677e-1 | 2.5805e-1 |
| | (2.07e-2)= | (1.33e-2)- | (1.39e-2)= | (4.03e-3)- | (2.14e-3)- | **(3.42e-2)**+ | (2.85e-2)- | (1.71e-3) |
| WFG9 | 3.3407e-1 | 3.2860e-1 | 3.3320e-1 | 3.3353e-1 | 3.3489e-1 | 3.2550e-1 | 3.3449e-1 | **3.3469e-1** |
| | (4.31e-4)- | (2.85e-3)- | (9.66e-4)- | (4.74e-4)- | (1.64e-3)= | (4.96e-2)- | (1.08e-3)- | **(5.33e-4)** |
| ±/= | 1/14/6 | 1/20/0 | 0/20/1 | 0/21/0 | 4/13/2 | 8/12/1 | 7/13/1 | |

discount factor $\gamma$=0.9. Algorithm execution terminated after 20,000 iterations, with mini-batch size set to 64 for DQN training updates.

To evaluate the performance of SFA-MOAHA in handling various complex PF, experiments are conducted on three benchmark test sets: ZDT, DTLZ, and WFG. To assess the performance of SFA-MOAHA, we present comparative experiments with five algorithms: Multi-objective Particle Swarm Optimization (MOPSO) [57], Non-Dominated Sorting Genetic Algorithm-II (NSGA-II) [46], Multi-objective grey wolf optimizer (MOGWO) [58], Non-Dominated Sorting Whale Optimization Algorithm (NSWOA) [59], Large-scale multi-objective optimization via reformulated decision variable analysis(LERD) [60], Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization(MOEADDQN) [44],and MOAHA. Table 1 displays the parameter settings for all comparison algorithms.

All experimental codes are run on MATLAB R2023b, which is installed on a computer with Windows 11 OS, an AMD Ryzen 5 3600 6-Core Processor (3.60 GHz), and 8GB RAM.

### 5.2. Rationality analysis of improvement strategies

To verify the rationality of improvement strategies for the MOAHA, common test functions (ZDT1, DTLZ2, and WFG1) are selected. A comparison is conducted among the original MOAHA and its two improved versions: S-MOAHA adopting with the spiral approximation strategy and F-MOAHA with the forward fusion learning strategy. Each algorithm runs independently for 30 times. Meanwhile, the performance of the three algorithms is measured using the average of two metrics, inverse generation distance (IGD) and hypervolume (HV). IGD evaluates convergence by computing the Euclidean distance to the true PF. HV assesses both convergence and diversity by computing the HV between the obtained Pareto front and the reference point (1,1).

Fig. 8 presents the IGD and HV convergence curves of MOAHA, F-MOAHA, and S-MOAHA on the ZDT1, DTLZ2, and WFG1 test problems. The results demonstrate clear performance patterns for each variant. In terms of the HV metric, S-MOAHA exhibits a slower initial rise and is ultimately outperformed by MOAHA in later iterations. Conversely, F-MOAHA surpasses MOAHA toward the end of the optimization process. Regarding the IGD metric, S-MOAHA converges rapidly during the early phase and maintains superior convergence performance compared to MOAHA thereafter. Although F-MOAHA converges more slowly initially, it also achieves better convergence than MOAHA in later stages.

These behavioral differences stem from the distinct mechanisms introduced in each algorithm. S-MOAHA incorporates an omnidirectional adaptive spiral local search, which enhances global exploration. When combined with its adaptive step-size adjustment mechanism, the algorithm effectively avoids local optima, leading to improved convergence as reflected in the IGD metric. F-MOAHA utilizes a forward fusion learning foraging strategy based on PF dominance levels. This approach leverages historical population information to strengthen exploratory

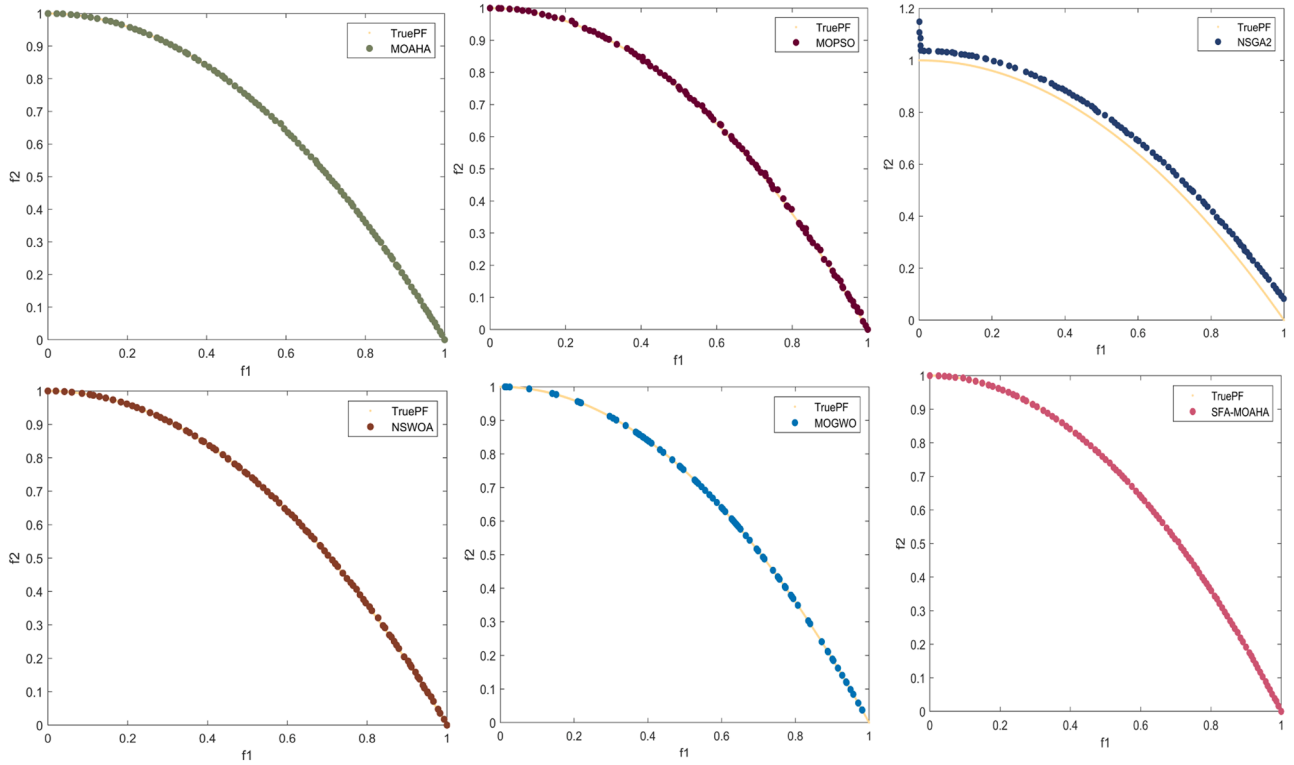



**Fig. 9.** Best PF obtained by all comparison algorithms for ZDT2.

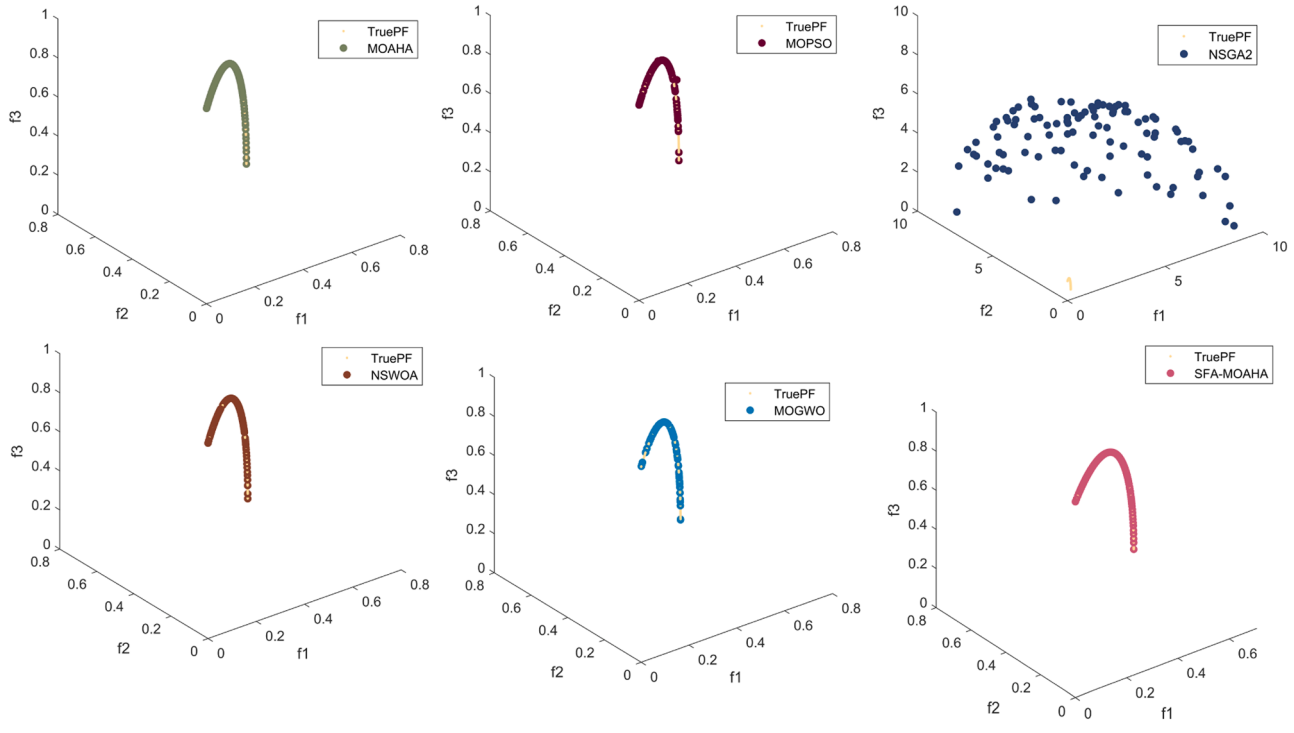



**Fig. 10.** Best PF obtained by all comparison algorithms for DTLZ6.

capability and increase population diversity, which explains its stronger HV performance in later iterations.

Tables 2 and 3 further shows that SFA-MOAHA, which integrates both strategies, achieves the best overall performance across all benchmark functions. This superior result is attributed to its DQN-based adaptive strategy selection mechanism. The mechanism dynamically chooses the most appropriate search strategy based on the current state, thereby effectively balancing convergence speed with population diversity.

In summary, the omnidirectional adaptive spiral local search primarily enhances convergence, while the forward fusion learning foraging strategy promotes diversity maintenance. The DQN-based

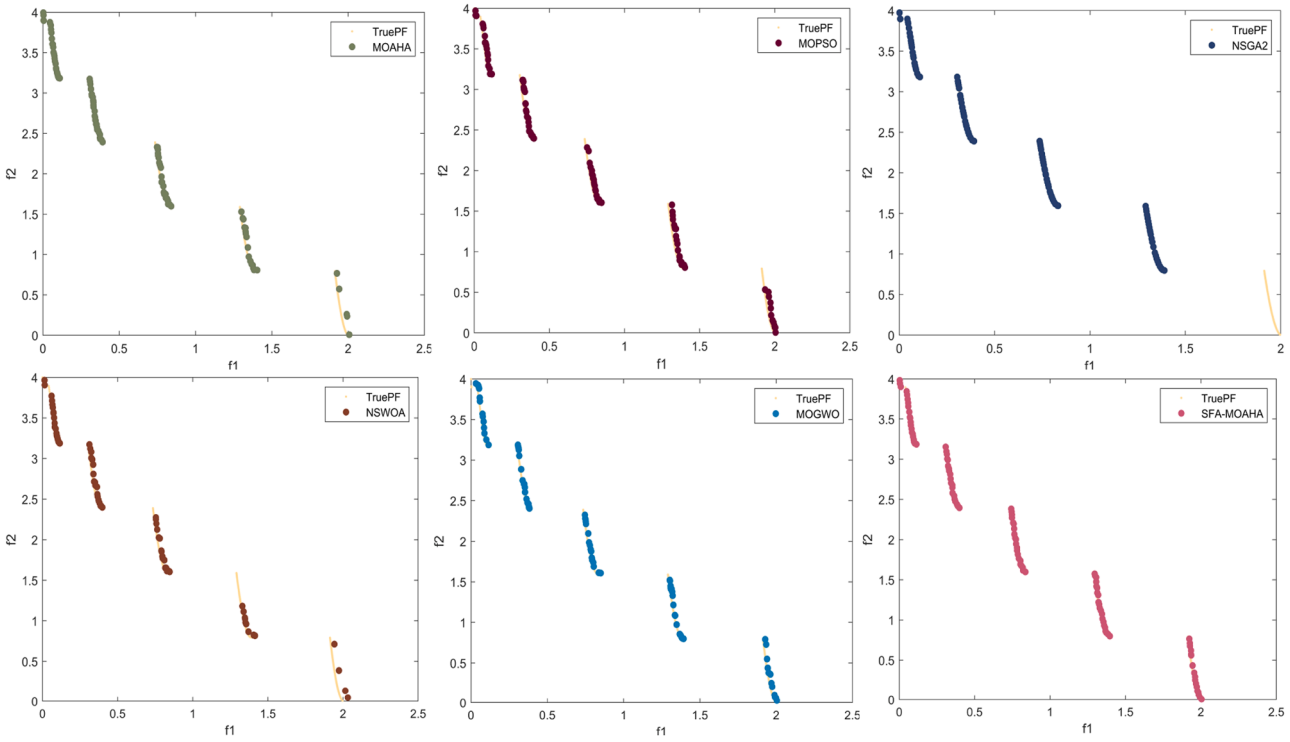



**Fig. 11.** Best PF obtained by all comparison algorithms for WFG2.

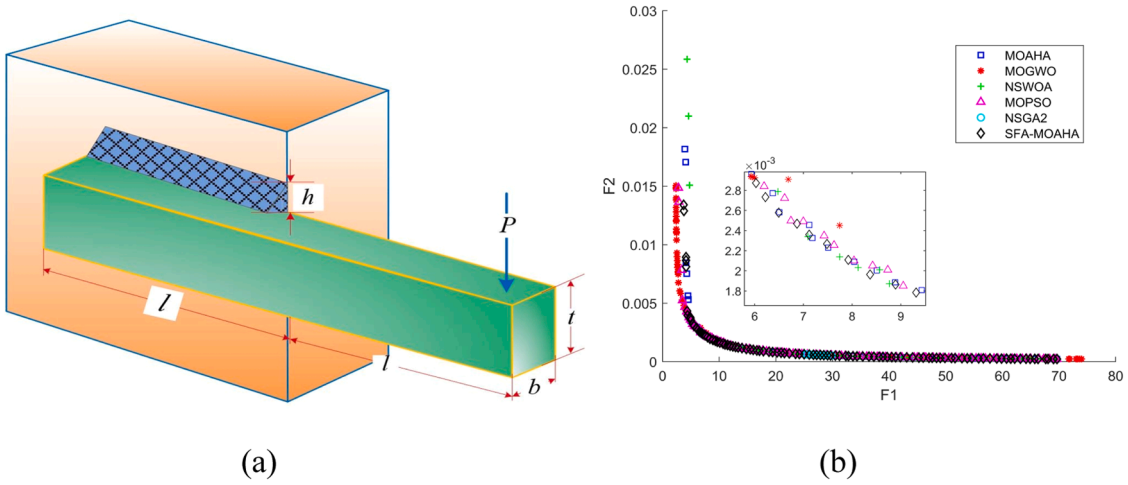



**Fig. 12.** Welded beam design: (a) welded beam design structural diagram, (b) optimization results.

selection mechanism allows SFA-MOAHA to adaptively balance these two aspects during different search phases. Collectively, these improvements significantly enhance the comprehensive performance of MOAHA, validating the effectiveness of the proposed methodological enhancements.

### 5.3. Experimental results and analysis of SFA-MOAHA

The maximum functional evaluation is set to 30,000 and the population size is 100 for all algorithms.

Table 2 and Table 3 list the mean and standard deviation of the IGD and HV derived from 30 independent operations of each algorithm. In these tables, the optimum outcome is emphasized for each row. The '+', '-', and '=' signify significantly better, worse, or similar to SFA-MOAHA, respectively. Additionally, statistical analysis is conducted using the Wilcoxon rank-sum test with a significance level of 0.05.

As shown in the results, SFA-MOAHA achieves the best IGD metric (Table 2) in 9 out of 21 test problems, performing significantly better than the other compared algorithms. MOEADDQN and LERD follow closely, with LERD algorithm showing strength on WFG problems and MOEADDQN demonstrating clear advantages on DZDT problems. In terms of the HV metric (Table 3), SFA-MOAHA, MOEADDQN, and LERD algorithm again rank among the top performers, obtaining the best results in 11, 6, and 4 test problems respectively. Notably, SFA-MOAHA maintains leading performance across both ZDT and WFG problems. Overall, compared to the deep reinforcement learning based MOEADDQN and the recent LERD algorithm, SFA-MOAHA exhibits comprehensive and robust competitive advantages. These additional comparisons clearly show that our proposed method outperforms previous leading optimization approaches. Besides IGD and HV

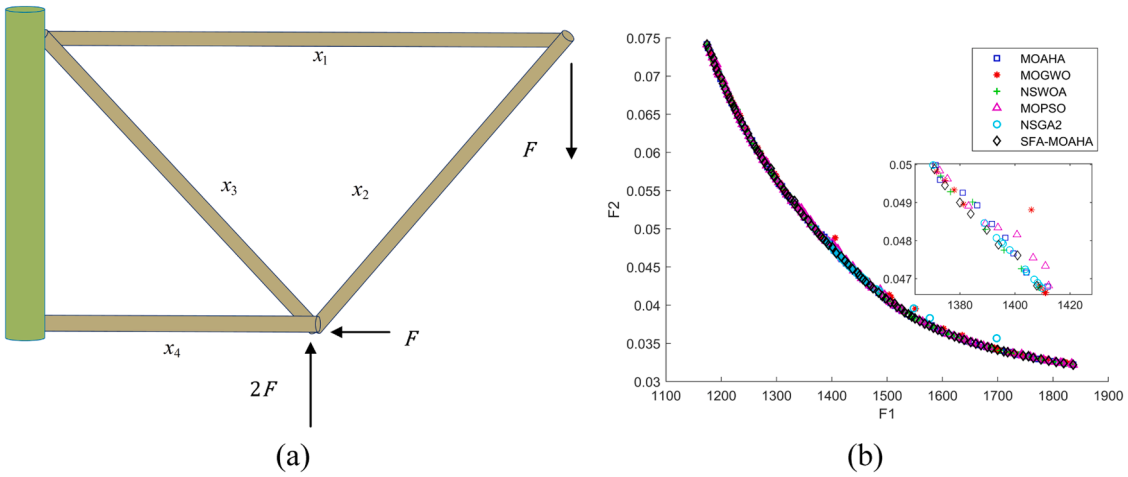



**Fig. 13.** Four-bar truss design: (a) four-bar truss structure diagram. (b) optimization results.

**Table 4**
Results of HV metric for four engineering problems.

| Engineering design problems | MOAHA | MOGWO | NSWOA | MOPSO | NSGA-II | SFA-MOAHA |
|---|---|---|---|---|---|---|
| Welded beam design | 0.945 (0.021)- | 0.852 (0.031)- | 0.852 (0.237)- | 0.921 (0.025)- | 0.531 (0.214)- | 0.997 (0.014) |
| Four-bar truss design | 0.912 (0.004)- | 0.922 (0.051)- | 0.763 (0.039)- | 0.853 (0.102)- | 0.721 (0.359)- | 0.994 (0.007) |

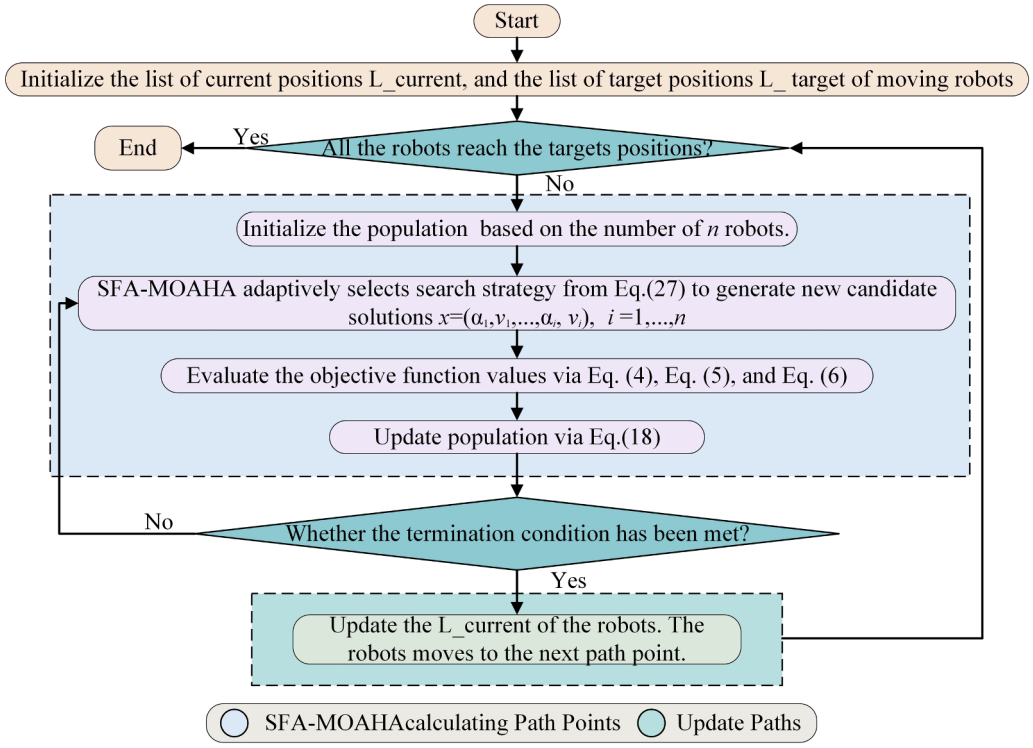



**Fig. 14.** Flowchart of SFA-MOAHA to solve MRPP.

comparisons, Wilcoxon rank-sum tests in Tables 2 and 3 show that SFA-MOAHA achieves the highest average score, demonstrating its superior overall performance.

Figs. 9–11 visually contrasts the performance of the various algorithms on the baseline issues. The problems are ZDT2, DTLZ6, and WFG2, which have non-linear, irregular, and discontinuous PFs similar to real-world complex scenarios. Specifically, Fig. 9 shows the experimental results for ZDT2. In this case, MOAHA and SFA-MOAHA have better convergence performance and PF distribution than other algorithms. As shown in Fig. 10 for DTLZ6, both MOAHA and SFA-MOAHA achieve optimal convergence and PF distribution. Fig. 11 depicts the experimental results for WFG2, SFA-MOAHA shows excellent

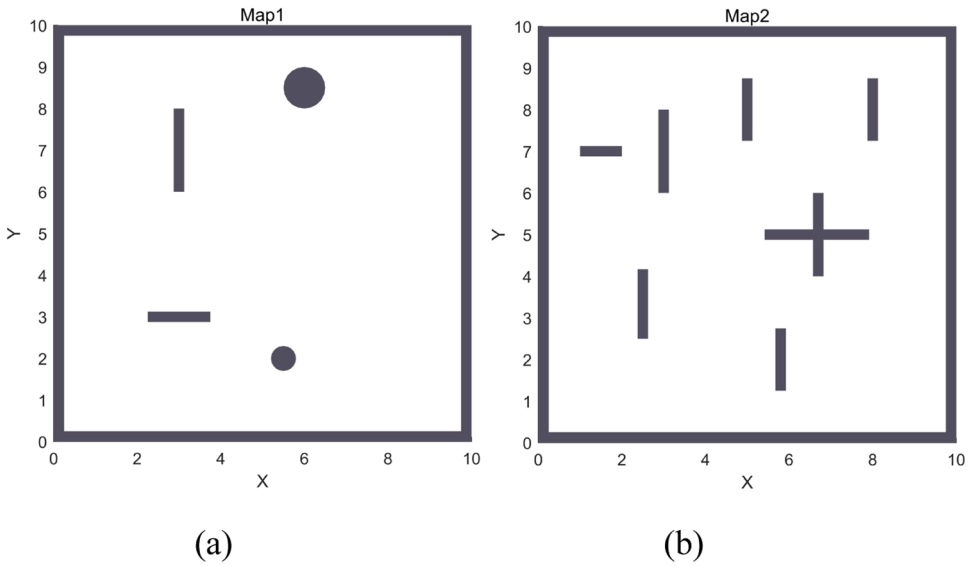



**Fig. 15.** Schematic diagram of the simulation map. (a) Map1. (b) Map2.

**Table 5**
A comparison of the average path length and the average number of steps between four and eight robots in Map 1.

| Number of robots | Robot | MOAHA | MOGWO | NSWOA | MOPSO | NSGA-II | SFA-MOAHA |
|---|---|---|---|---|---|---|---|
| 4 robots | R1 | 10.4587 | 14.6769 | 11.8099 | 11.1562 | 11.7785 | 10.4714 |
| | R2 | 7.7325 | 10.1283 | 8.0173 | 9.6325 | 7.4677 | 7.3786 |
| | R3 | 7.9921 | 9.5035 | 8.1712 | 8.0242 | 7.0245 | 6.4716 |
| | R4 | 8.2904 | 9.4737 | 9.9646 | 8.4986 | 7.7512 | 8.0049 |
| | Total APL | 34.4737(3) | 43.7824(6) | 37.963(5) | 37.3115(4) | 34.0218(2) | **32.3266(1)** |
| | Total ANS | 38 | 63 | 48 | 47 | 45 | **36** |
| 8 robots | R1 | 12.011 | 17.1532 | 12.8742 | 13.6025 | 12.4089 | 12.2351 |
| | R2 | 7.1222 | 9.7459 | 8.6978 | 9.5765 | 9.0525 | 7.6424 |
| | R3 | 12.6756 | 15.3857 | 13.4971 | 16.3555 | 13.1619 | 12.6705 |
| | R4 | 11.9428 | 14.1259 | 8.6982 | 12.8553 | 11.9019 | 9.4601 |
| | R5 | 6.4586 | 22.5435 | 7.131 | 8.1282 | 10.1817 | 6.3519 |
| | R6 | 7.1622 | 13.7999 | 6.7828 | 7.8751 | 6.5538 | 5.1351 |
| | R7 | 7.4895 | 8.1574 | 6.3305 | 12.0714 | 6.8364 | 5.6647 |
| | R8 | 7.2027 | 6.2221 | 7.1657 | 11.5459 | 6.6313 | 7.6701 |
| | Total APL | 72.0646(3) | 107.1337(6) | 71.1773(2) | 92.0105(5) | 76.7283(4) | **66.8299(1)** |
| | Total ANS | 85 | 160 | 90 | 148 | 112 | **76** |

performance in terms of both convergence speed and PF distribution compared to other algorithms. It validates the effectiveness of DQN for search strategy selection.

The experimental results confirm that SFA-MOAHA achieves superior performance by effectively balancing exploration and exploitation, demonstrating the advantages of DQN in search strategy selection. Compared with other algorithms, the SFA-MOAHA demonstrates superior convergence and population diversity, further validating the effectiveness of the proposed forward fusion learning strategy based on PF dominance levels and the integration of spiral approximation strategy and omnidirectional flight skill.

### 5.4. SFA-MOAHA for multi-objective engineering design problems

To assess the validity of SFA-MOAHA in actual MOOPs, two common engineering design problems are chosen as benchmarks. Before testing, parameters for these problems are set: 300 for the maximum iterations, 100 for both the population and archive sizes. Other experimental conditions and parameters remain consistent with those of the previous settings in Section 5.3. The SFA-MOAHA is compared with the five algorithms presented in Section 4.1. During the evaluation phase, each algorithm undergoes 30 separate trials per benchmark problem, with performance metrics averaged across runs.

**Welded beam design:** The welded beam design aims to minimize fabrication cost and deflection simultaneously. As indicated in Fig. 12a [61], this optimization issue involves four design variables and four constraints, formulated as follows:

Design variable:

$$x == [h, l, t, b] = [x_1, x_2, x_3, x_4], \tag{30}$$

Objectives:

$$\text{Minimize} f_1(x) = 1.10471 x_1^2 x_2 + 0.04811 x_3 x_4 (14 + x_2), \tag{31}$$

$$\text{Minimize} f_2(x) = \frac{4PL^3}{E x_3^3 x^4}, \tag{32}$$

Variable ranges:

$$0.1 \le x_1 \le 2, \quad 0.1 \le x_2 \le 10, \quad 0.1 \le x_3 \le 10, \quad 0.1 \le x_4 \le 2, \tag{33}$$

Subject to:

$$g_1(x) = \tau(\vec{x}) - \tau_{\max} \le 0, g_2(x) = P - P_c(\vec{x}) \le 0, g_3(x) = \sigma(\vec{x}) - \sigma_{\max} \le 0, g_4(\vec{x}) = x_1 - x_4 \le 0, \tag{34}$$

where $\tau(\vec{x}) = \sqrt{(\tau')^2 + 2\tau'\tau'' x_2/2R + (\tau'')^2}$, $\tau' = P/\sqrt{2}x_1 x_2$ $\tau'' = MR/J$,

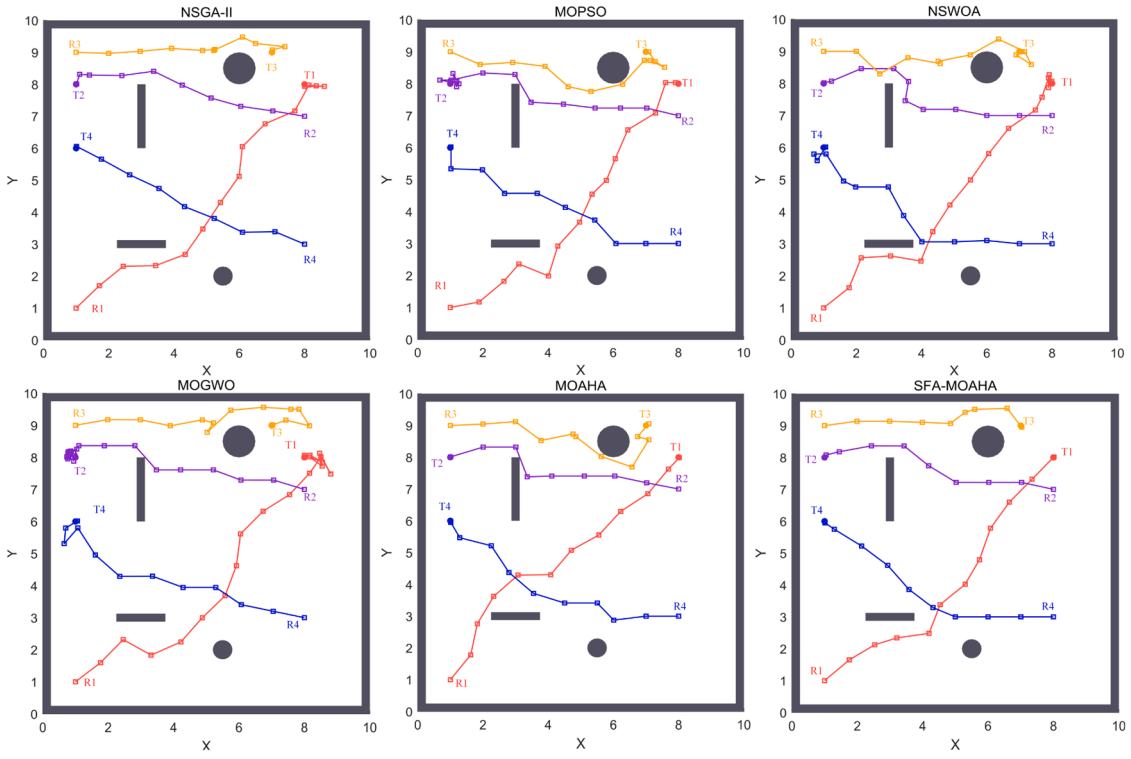



**Fig. 16.** Four robot paths planning in Map 1.

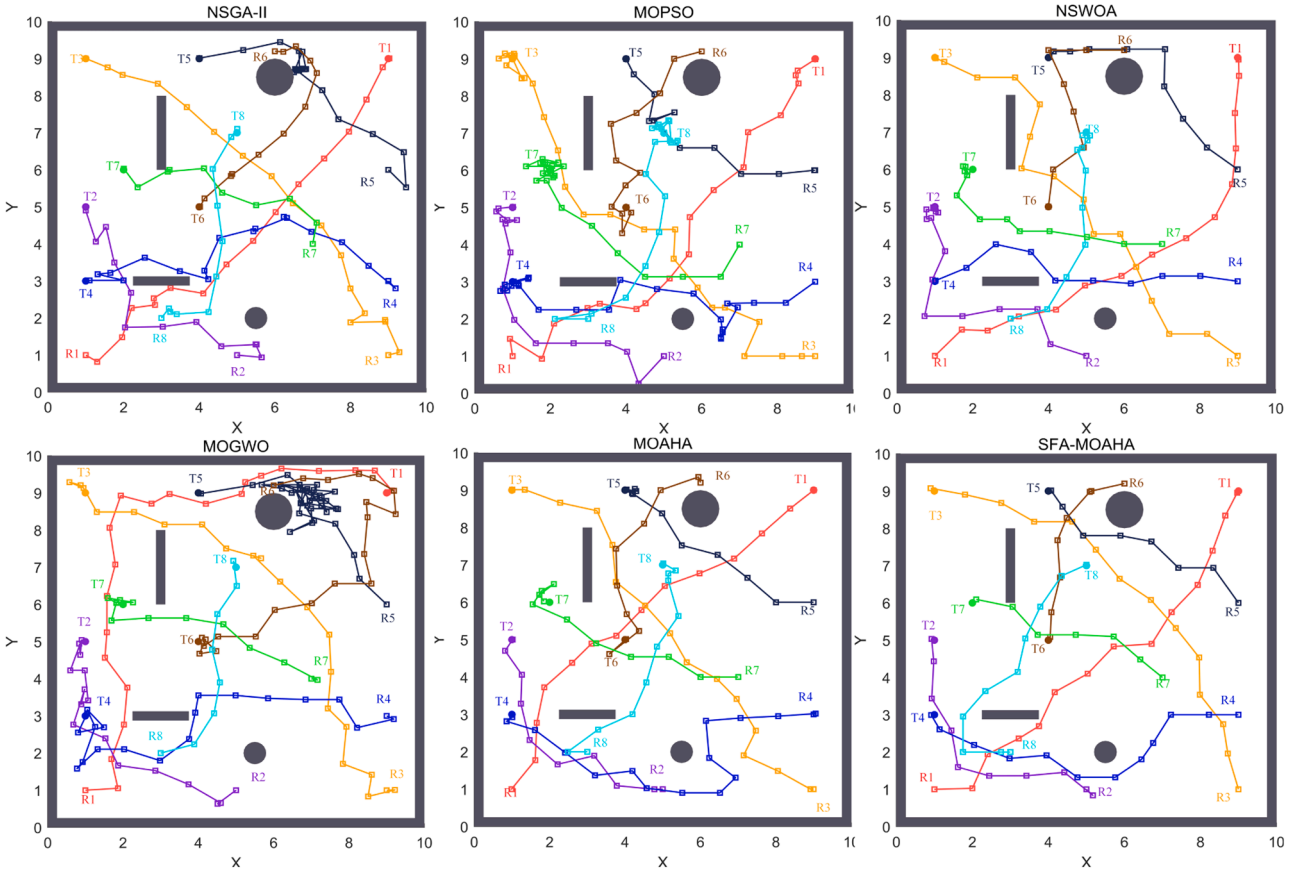



**Fig. 17.** Eight robot paths planning in Map 1.

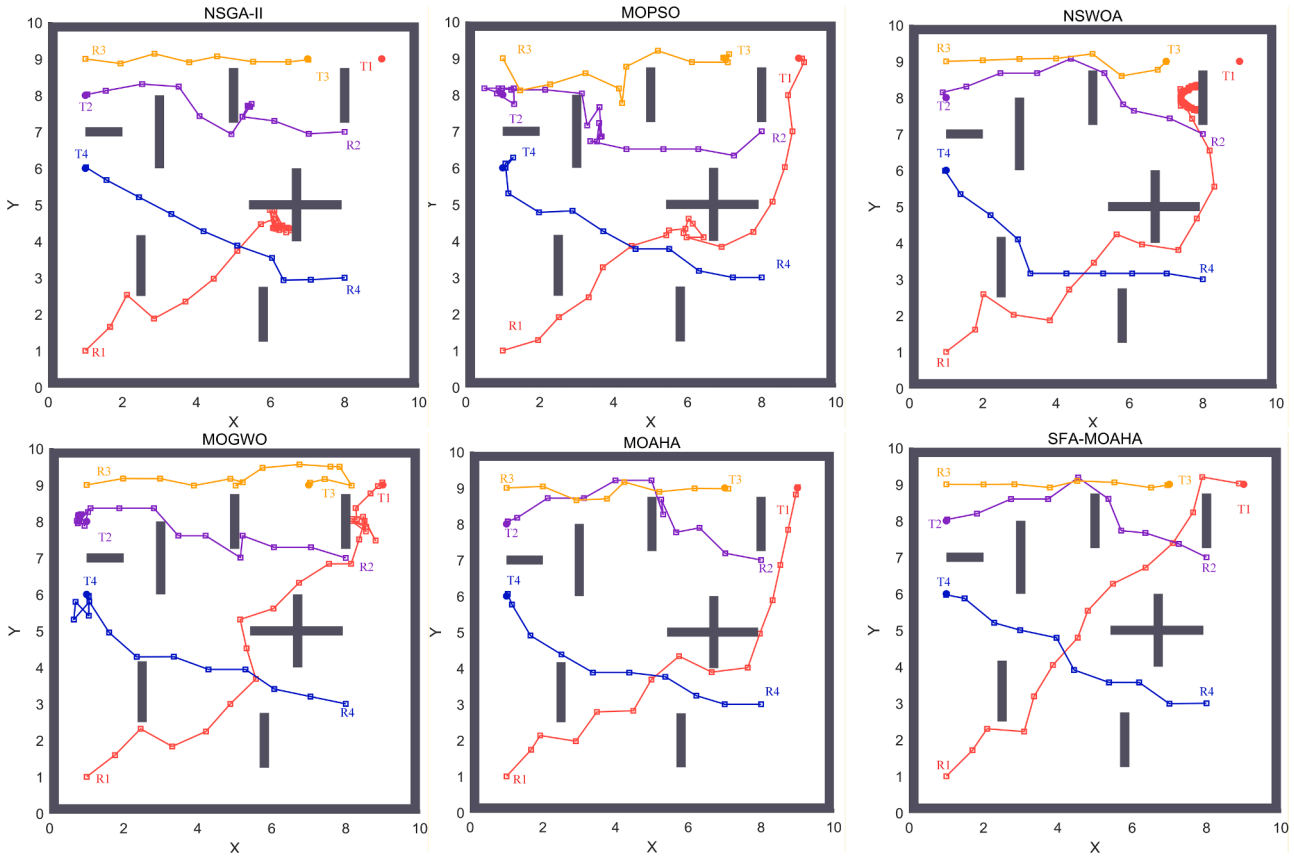



**Fig. 18.** Four robot paths planning in Map 2.

$$M = P(L + x_2/2),\quad R = \sqrt{\frac{x_2^2}{4} + \left(\frac{x_1+x_3}{2}\right)^2},\quad J = 2\left\{\sqrt{2}x_1x_2\left[\frac{x_2^3}{4} + \left(\frac{x_1+x_3}{2}\right)^2\right]\right\},\ \sigma(\vec{x}) = \frac{6PL}{x_4x_3^2},\ P_c(\vec{x}) = \frac{4.013E\sqrt{\frac{x_3^2x_4^6}{36}}}{L^2}\left(1 - \frac{x_3}{2L}\sqrt{\frac{E}{4G}}\right),$$

As illustrated in Fig. 12b, all algorithms can converge well to solve the welded beam design. SFA-MOAHA shows uniform distribution and good convergence and coverage, which confirms the feasibility of SFA-MOAHA in the design of welded beams.

**Four-bar truss design:** The four-bar truss design aims to minimize the volume and joint displacement [62]. As presented in Fig. 13a, this problem has 4 design variables. The details are depicted as follows:

Design variable:

$$x = [x_1, x_2, x_3, x_4], \tag{35}$$

Objectives:

$$\text{Minimize} f_1(x) = L\left(2x_1 + \sqrt{2x_2} + \sqrt{x_3} + x_4\right), \tag{36}$$

$$\text{Minimize} f_2(x) = \frac{FL}{E}\left(2 \Big/ x_2 + 2\sqrt{2} \Big/ x_2 + 2\sqrt{2} \Big/ x_3 + 2 \Big/ x_4\right), \tag{37}$$

where $F = 10$, $E = 2 \times 10^5$, $L = 200$.

Variable ranges:

$$1 \le x_1,\ \sqrt{2} \le x_2,\ x_3 \le 3,\ x_4 \le 3, \tag{38}$$

As revealed in Fig. 13b, NSGA-II converges poorly. In contrast, MOAHA, MOGWO, MOPSO, NSWOA and SFA-MOAHA not only converge well but also have good coverage. Among them, SFA-MOAHA is evenly distributed, which confirms its feasibility in four-bar truss design.

HV is employed to assess solution quality in engineering design problems, as it provides integrated metrics for both convergence and diversity [63]. The HV results for the welded beam design and four-bar linkage design are summarized in Table 4. The outcomes indicate that SFA-MOAHA achieves highly competitive performance on both engineering design problems. For the welded beam and four-bar beam cases, SFA-MOAHA yields significantly higher HV values than the other compared algorithms, demonstrating statistically notable differences. Attaining the maximum HV value, SFA-MOAHA confirms its ability to deliver well-balanced solutions with strong convergence and diversity.

## 6. Analysis of optimal performance for MRPP problem

To assess the efficacy of the suggested SFA-MOAHA in addressing the MRPP issue formulated in Section 2, we perform a series of comparative experiments. These experiments are conducted under the experimental conditions and parameters defined in Section 4, where five other algorithms are also introduced for comparison. Fig. 14 presents the flowchart for MRPP resolution.

To assess its performance in MRPP, two 10m×10 m environment maps of different complexities (Map 1 in Fig. 15a and Map 2 in Fig. 15b) are designed. In the simulation, the mobile robot is represented by a square with a side length of 0.25 m, different colors are used to represent different paths, and the starting and destination points of the robot are preset. Obstacles are gray, and the complexity of the workspace relies on the size and quantity of obstacles. Different-colored squares denote the positions of robots, and circles centered on the target points represent the target areas. When a robot reaches the target area of it, path planning concludes, and the color of each robot matches the start and target points of it.

All algorithms have a population size of $SN$= 100 and a termination condition of $max_FES$= 30,000. The rotation angle of the robot ranges from $-\pi$ to $\pi$, and its speed ranges from 0 to 1. Each algorithm is inde-

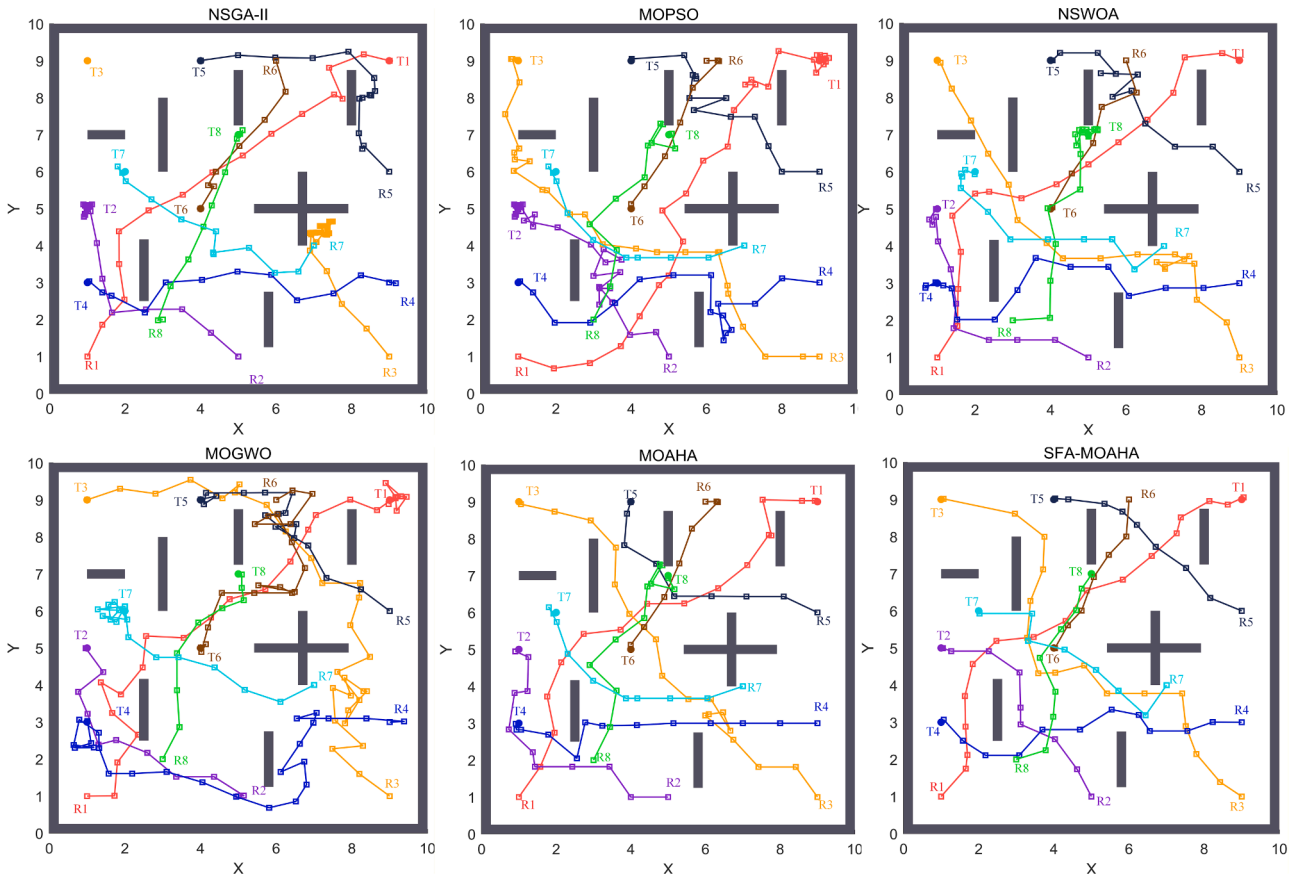



**Fig. 19.** Eight robot paths planning in Map 2.

**Table 6**
A comparison of the average path length and the average number of steps between four and eight robots in Map 2.

| Number of robots | Robot | MOAHA | MOGWO | NSWOA | MOPSO | NSGA-II | SFA-MOAHA |
|---|---|---|---|---|---|---|---|
| 4 robots | R1 | 13.6625 | 16.8965 | - | 14.9515 | - | 12.4793 |
| | R2 | 9.3402 | 10.7463 | 8.1685 | 12.971 | 8.7937 | 8.1853 |
| | R3 | 6.4098 | 9.2271 | 6.3109 | 8.5484 | 6.1255 | 5.994 |
| | R4 | 8.1296 | 10.1144 | 8.4864 | 8.9377 | 7.9504 | 8.087 |
| | Total APL | 37.5421(2) | 46.9842(4) | - | 45.4085(3) | - | **34.7456(1)** |
| | Total ANS | 42 | 70 | - | 64 | - | **38** |
| 8 robots | R1 | 13.4387 | 17.5903 | 13.5412 | 16.3171 | 13.3764 | 12.8643 |
| | R2 | 7.9411 | 7.6244 | 7.5482 | 12.65 | 8.1529 | 6.4608 |
| | R3 | 14.3653 | 21.4957 | 14.7507 | 15.0355 | - | 14.3156 |
| | R4 | 9.2584 | 17.1733 | 10.774 | 12.2157 | 9.4862 | 9.0519 |
| | R5 | 7.1145 | 10.3762 | 8.755 | 9.0664 | 8.206 | 6.0723 |
| | R6 | 4.9088 | 11.8144 | 4.9499 | 4.9088 | 5.0013 | 4.6031 |
| | R7 | 8.115 | 6.0197 | 8.4373 | 8.1847 | 5.8861 | 6.0944 |
| | R8 | 6.7554 | 10.5448 | 7.5667 | 6.7554 | 7.4603 | 6.9207 |
| | Total APL | 71.8973(2) | 102.6389(5) | 76.3229(3) | 85.1336(4) | - | **66.3831(1)** |
| | Total ANS | 89 | 146 | 104 | 133 | - | **79** |

pendently run 30 times on each map. Tables 4 and 5 present the average number of steps (ANS) and the average path length (APL) for each robot to reach the target, and rank the algorithms according to the total APL across all tasks.

Frist, the map 1 in Fig. 15a, with low complexity and four differently-shaped obstacles, is chosen for the experiments. Two test scenarios are set up: one with 4 robots (Fig. 16) and the other with 8 robots (Fig. 17). The main simulation results are shown in Table 5 and Figs. 16 and 17. Comparing the results in Table 5 shows the validity of the SFA-MOAHA. The paths planned by SFA-MOAHA are 6.23 % and 7.26 % shorter than those planned by MOAHA in the path planning tasks with 4- and 8-robots, respectively. It outperforms other algorithms in ANS and APL, especially in APL where the other five algorithms are significantly inferior. The number of steps indicates algorithm performance. In MRPP problem with variable robot speeds, fewer steps mean shorter paths and faster speeds. The SFA-MOAHA excels here. Figs. 16 and 17 show all algorithms can guide robots to targets, validating the feasibility of the MRPP model. The figures reveal the MOGWO and MOPSO need many steps near targets, indicating that they are likely to be stuck in a local optimum with limited ability to locate the optimal path. In contrast, the SFA-MOAHA rarely gets stuck in local optima during 4-and 8-robot path planning, indicating its strong capabilities of exploration and exploitation and reconfirming its superiority in MRPP problem.

Then, the map 2 in Fig. 15b, featuring high complexity and seven distinct obstacles, is selected for the experiments. Two test scenarios are established: one with 4 robots (Fig. 18) and the other with 8 robots

**Table 7**
Compared the average path length and average number of steps for the 8 robot paths between MOEADDQN and LERD in Map 2.

| Number of robots | Robot | MOEADDQN | LERD | SFA-MOAHA |
|---|---|---|---|---|
| 8 robots | R1 | 12.7491 | 13.479 | 12.8643 |
| | R2 | 6.9213 | 6.8162 | 6.4608 |
| | R3 | 15.1873 | 16.1823 | 14.3156 |
| | R4 | 9.5325 | 9.8731 | 9.0519 |
| | R5 | 6.0396 | 6.4786 | 6.0723 |
| | R6 | 5.1732 | 4.3851 | 4.6031 |
| | R7 | 6.0025 | 6.4901 | 6.0944 |
| | R8 | 6.4514 | 6.4147 | 6.9207 |
| | Total APL | 68.0569(2) | 70.1191(3) | **66.3831(1)** |
| | Total ANS | 83 | 86 | **79** |

(Fig. 19). The key simulation results are presented in Table 6 and Figs. 18 and 19. The outcomes in Table 6 illustrate the validity of the SFA-MOAHA. The paths located by SFA-MOAHA are 7.45 % and 7.67 % shorter than MOAHA in the path planning tasks with 4- and 8- robots, respectively. The SFA-MOAHA outperforms other algorithms in terms of ANS and APL, achieving the best results in both metrics.

In the MRPP problem with variable robot speeds, fewer steps imply shorter paths and faster speeds. The SFA-MOAHA performs excellently in this regard. As shown in Figs. 18 and 19, in both 4-robot and 8-robot scenarios, NSGA-II and NSWOA converge to local optima and fail to guide robots to their targets. Meanwhile, MOGWO and MOPSO generate longer paths and require more steps. This limitation arises from the absence of an adaptive selection mechanism in these algorithms, which prevents trapped robots from escaping local optima. Consequently, both the exploitation capability and the accuracy of target localization are reduced, leading to a significant increase in path length and number of steps. The SFA-MOAHA rarely gets stuck in local optima during 4- and 8-robot path planning. This indicates its strong exploration and exploitation capabilities and reaffirms its superiority in solving the MRPP problem in complex environments.

To further strengthen the empirical validation, comparison experiments are conducted with advanced MRPP algorithms MOEADDQN and LERD for path planning for 8 robots on Map2. The results are summarized in Table 7. As shown in Table 7, SFA-MOAHA achieves the shortest paths for robots 2, 3, 4, 6, and 8, while obtaining the minimum values in both APL and ANS. These results show that SFA-MOAHA maintains clear advantages over the advanced MRPP algorithms, confirming its effectiveness and competitiveness in complex environments.

As shown in Fig. 20, the DQN component in SFA-MOAHA introduces significant computational overhead. Specifically, the action selection phase requires <1 s, parameter update takes approximately 10 s, and the initial training exhibits the longest duration exceeding 80 s. In terms of time proportion, the training phase is the costliest, followed by parameter update and action selection. This considerable overhead currently limits the real-time applicability of the algorithm in highly dynamic or resource-constrained scenarios. Therefore, further optimization is necessary to reduce computational costs for practical deployment.

## 7. Conclusions

This paper addresses the online MRPP problem by proposing an improved algorithm named SFA-MOAHA. The algorithm integrates a spiral approximation strategy to enhance local exploitation and a forward fusion learning strategy to promote global exploration. A DQN is employed as an adaptive strategy scheduler, enabling the algorithm to dynamically select the most suitable search scheme at each iteration. The results demonstrate a clear advantage for SFA-MOAHA over the original MOAHA and other algorithms across all tested scenarios, including benchmarks and complex simulations, with improvements in both path length and step size. Although SFA-MOAHA demonstrates strong performance, its current implementation uses fixed DQN parameters and the simulation environment is relatively simple. In future work, we plan to validate the algorithm in more challenging 3D scenarios, analyze key DQN hyperparameter sensitivity, and incorporate advanced planning mechanisms for larger robotic swarms.

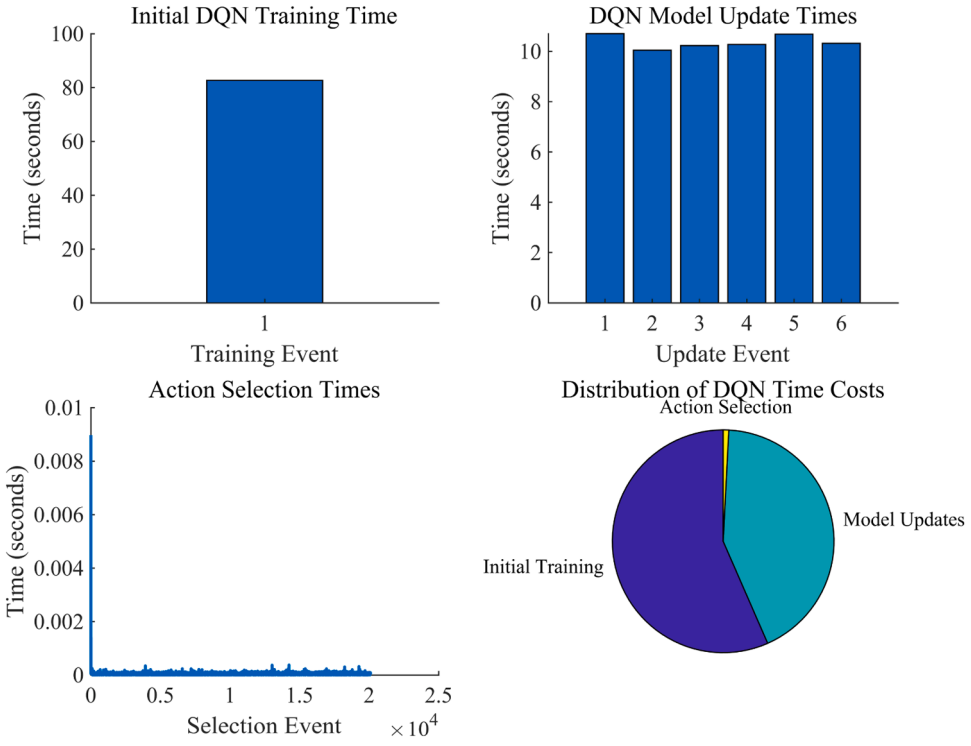



**Fig. 20.** DQN component computational overhead.

## CRediT authorship contribution statement

**Ao Ren:** Writing – original draft, Validation, Methodology, Investigation, Formal analysis, Conceptualization. **Junxing Zhang:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Investigation, Formal analysis. **Jiaxue Ran:** Resources, Methodology, Investigation. **Jiewen Pang:** Software, Resources, Investigation. **Lunjie Wang:** Writing – review & editing, Investigation.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

This work is supported in part by National Natural Science Foundation of China (52465059), Guizhou Provincial Science and Technology Support Program Projects ([2025]049), in part by the Science and Technology Incubation Planning Project of Guizhou University ([2020] 75), in part by the Guizhou University Talent Fund ([2022]27),in part by the Foundation of Key Laboratory of Advanced Manufacturing Technology, Ministry of Education, Guizhou University (GZUAMT2021KF [11]).

## Data availability

No data is used for the research described in the article.

## References

[1] S.A. Akgun, M. Ghafurian, M. Crowley, K. Dautenhahn, Using affect as a communication modality to improve human-robot communication in robot-assisted search and rescue scenarios, IEEE Trans. Affective Comput. 14 (2023) 3013–3030.
[2] G. Yang, S. Wang, H. Okamura, T. Yasui, S. Ino, K. Okuhata, Y. Mizobuchi, Autonomous cart docking for transportation robots in complex and dynamic construction environments, Autom. Constr. 171 (2025) 105968.
[3] M. Tutam, R. De Koster, To walk or not to walk? Designing intelligent order picking warehouses with collaborative robots, Transp. Res. E 190 (2024) 103696.
[4] Z. Zhao, J. Lyu, Y. Chu, K. Liu, D. Cao, C. Wu, L. Qin, S. Qin, Toward generalizable robot vision guidance in real-world operational manufacturing factories: a semi-supervised knowledge distillation approach, Robot. Comput. Integr. Manuf. 86 (2024) 102639.
[5] H. Cao, J. Xu, Z. Yang, L. Shangguan, J. Zhang, X. He, Y. Liu, Scaling up edge-assisted real-time collaborative visual SLAM applications, IEEE ACM Trans. Netw. 32 (2024) 1823–1838.
[6] C. Ntakolia, D.V. Lyridis, A comparative study on ant colony optimization algorithm approaches for solving multi-objective path planning problems in case of unmanned surface vehicles, Ocean Eng. 255 (2022) 111418.
[7] T.I. Zohdi, A machine-learning enabled digital-twin framework for tactical drone-swarm design, Comput. Methods Appl. Mech. Eng. 442 (2025) 117999.
[8] Z. Wu, J. Dai, B. Jiang, H.R. Karimi, Robot path planning based on artificial potential field with deterministic annealing, ISA Trans. 138 (2023) 74–87.
[9] Y. Yang, W. Merkt, V. Ivan, Z. Li, V. Vijayakumar, HDRM: a resolution complete dynamic roadmap for real-time motion planning in complex scenes, IEEE Robot. Autom. Lett. 3 (2018) 551–558.
[10] K. Alton, I.M. Mitchell, Fast marching methods for stationary Hamilton–Jacobi equations with axis-aligned anisotropy, SIAM J. Numer. Anal. 47 (2009) 363–385.
[11] E. Auh, J. Kim, Y. Joo, J. Park, G. Lee, I. Oh, N. Pico, H. Moon, Unloading sequence planning for autonomous robotic container-unloading system using A-star search algorithm, Eng. Sci. Technol. Int. J. 50 (2024) 101610.
[12] L. Deng, S. Liu, A multi-strategy improved slime mould algorithm for global optimization and engineering design problems, Comput. Methods Appl. Mech. Eng. 404 (2023) 115764.
[13] B. Shu, G. Hu, M. Cheng, C. Zhang, MSFPSO: multi-algorithm integrated particle swarm optimization with novel strategies for solving complex engineering design problems, Comput. Methods Appl. Mech. Eng. 437 (2025) 117791.
[14] M. Abdel-Basset, R. Mohamed, M. Abouhawwash, Fungal growth optimizer: a novel nature-inspired metaheuristic algorithm for stochastic optimization, Comput. Methods Appl. Mech. Eng. 437 (2025) 117825.
[15] F. Wu, S. Li, J. Zhang, R. Xie, M. Yang, Bernstein-based oppositional-multiple learning and differential enhanced exponential distribution optimizer for real-world optimization problems, Eng. Appl. Artif. Intell. 138 (2024) 109370.
[16] F. Wu, J. Zhang, S. Li, D. Lv, M. Li, An enhanced differential evolution algorithm with Bernstein operator and refracted oppositional-mutual learning strategy, Entropy 24 (2022) 1205.
[17] S. Wang, J. Zhang, S. Li, F. Wu, S. Li, An enhanced slime mould algorithm with triple strategy for engineering design optimization, J. Comput. Des. Eng. 11 (2024) 36–74.
[18] M.N.A. Wahab, S. Nefti-Meziani, A. Atyabi, A comparative review on mobile robot path planning: classical or meta-heuristic methods, Annu. Rev. Control 50 (2020) 233–252.
[19] S. Tian, Y. Li, Y. Kang, J. Xia, Multi-robot path planning in wireless sensor networks based on jump mechanism PSO and safety gap obstacle avoidance, Future Gener. Comput. Syst. 118 (2021) 37–47.
[20] F.H. Ajeil, I.K. Ibraheem, M.A. Sahib, A.J. Humaidi, Multi-objective path planning of an autonomous mobile robot using hybrid PSO-MFB optimization algorithm, Appl. Soft Comput. 89 (2020) 106076.
[21] S. Lin, A. Liu, J. Wang, X. Kong, An intelligence-based hybrid PSO-SA for mobile robot path planning in warehouse, J. Comput. Sci. 67 (2023) 101938.
[22] S. Lin, A. Liu, J. Wang, X. Kong, An improved fault-tolerant cultural-PSO with probability for multi-AGV path planning, Expert Syst. Appl. 237 (2024) 121510.
[23] P. Duan, Z. Yu, K. Gao, L. Meng, Y. Han, F. Ye, Solving the multi-objective path planning problem for mobile robot using an improved NSGA-II algorithm, Swarm Evol. Comput. 87 (2024) 101576.
[24] M. Nazarahari, E. Khanmirza, S. Doostie, Multi-objective multi-robot path planning in continuous environment using an enhanced genetic algorithm, Expert Syst. Appl. 115 (2019) 106–120.
[25] L. Wu, X. Huang, J. Cui, C. Liu, W. Xiao, Modified adaptive ant colony optimization algorithm and its application for solving path planning of mobile robot, Expert Syst. Appl. 215 (2023) 119410.
[26] C. Liu, L. Wu, W. Xiao, G. Li, D. Xu, J. Guo, W. Li, An improved heuristic mechanism ant colony optimization algorithm for solving path planning, Knowl. Based Syst. 271 (2023) 110540.
[27] W. Liang, M. Lou, Z. Chen, H. Qin, C. Zhang, C. Cui, Y. Wang, An enhanced ant colony optimization algorithm for global path planning of deep-sea mining vehicles, Ocean Eng. 301 (2024) 117415.
[28] Y. Cui, W. Hu, A. Rahmani, Multi-robot path planning using learning-based artificial bee colony algorithm, Eng. Appl. Artif. Intell. 129 (2024) 107579.
[29] M.Y. Yildirim, R. Akay, An efficient grid-based path planning approach using improved artificial bee colony algorithm, Knowl. Based Syst. 318 (2025) 113528.
[30] S. Fu, K. Li, H. Huang, C. Ma, Q. Fan, Y. Zhu, Red-billed blue magpie optimizer: a novel metaheuristic algorithm for 2D/3D UAV path planning and engineering design problems, Artif. Intell. Rev. 57 (2024) 134.
[31] M. Reda, A. Onsy, A.Y. Haikal, A. Ghanbari, A novel reinforcement learning-based multi-operator differential evolution with cubic spline for the path planning problem, Artif. Intell. Rev. 58 (2025) 142.
[32] C. Zhu, Y. Bouteraa, M. Khishe, D. Martín, F. Hernando-Gallego, T. Vaiyapuri, Enhancing unmanned marine vehicle path planning: a fractal-enhanced chaotic grey wolf and differential evolution approach, Knowl. Based Syst. 317 (2025) 113481.
[33] W. Zhao, L. Wang, S. Mirjalili, Artificial hummingbird algorithm: a new bio-inspired optimizer with its engineering applications, Comput. Methods Appl. Mech. Eng. 388 (2022) 114194.
[34] M. Hosseinzadeh, A.M. Rahmani, F.M. Husari, O.M. Alsalami, M. Marzougui, G. N. Nguyen, S.-W. Lee, A survey of artificial hummingbird algorithm and its variants: statistical analysis, performance evaluation, and structural reviewing, Arch. Comput. Methods Eng. 32 (2025) 269–310.
[35] W. Zhao, Z. Zhang, S. Mirjalili, L. Wang, N. Khodadadi, S.M. Mirjalili, An effective multi-objective artificial hummingbird algorithm with dynamic elimination-based crowding distance for solving engineering design problems, Comput. Methods Appl. Mech. Eng. 398 (2022) 115223.
[36] R.M. Abo-bakr, N. Mohamed, M.A. Eltaher, S. Emam, Multi-objective optimization for snap-through response of spherical shell panels, Appl. Math. Model. 127 (2024) 711–729.
[37] G. Li, X. Wei, H. Yang, A method for accurate prediction of photovoltaic power based on multi-objective optimization and data integration strategy, Appl. Math. Model. 136 (2024) 115643.
[38] M.S. Abid, H.J. Apon, I.M. Nafi, A. Ahmed, R. Ahshan, Multi-objective architecture for strategic integration of distributed energy resources and battery storage system in microgrids, J. Energy Storage 72 (2023) 108276.
[39] J. Xu, R. Wang, J. Mao, Z. Xiang, Fixed-time consensus of leader-following second-order nonlinear multi-agent systems via dynamic gains, Int. J. Robust Nonlinear Control 35 (2025) 7837–7849.
[40] J. Xu, R. Wang, J. Mao, Z. Xiang, New results on predefined-time consensus for nonlinear multi-agent systems under Lipschitz condition: a dynamic-gains-based method, J. Franklin Inst. 362 (2025) 107894.
[41] F. Wu, S. Li, J. Zhang, L. Yu, X. Xiong, L. Wu, Q-learning-based exponential distribution optimizer with multi-strategy guidance for solving engineering design problems and robot path planning, Results Eng. 27 (2025) 105705.
[42] T. Li, Y. Meng, L. Tang, Scheduling of continuous annealing with a multi-objective differential evolution algorithm based on deep reinforcement learning, IEEE Trans. Autom. Sci. Eng. 21 (2024) 1767–1780.
[43] K. Li, T. Zhang, R. Wang, Deep reinforcement learning for multiobjective optimization, IEEE Trans. Cybern. 51 (2021) 3103–3114.
[44] Y. Tian, X. Li, H. Ma, X. Zhang, Y. Jin, Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization, IEEE Trans. Emerg. Top. Comput. Intell. 7 (2023) 1051–1064.

[45] T.C. Service, A no free lunch theorem for multi-objective optimization, Inf. Process. Lett. 110 (2010) 917–923.
[46] Y. Zhao, J. Lu, Q. Yan, L. Lai, L. Xu, Research on cell manufacturing facility layout problem based on improved NSGA-II, Comput. Mater. Contin. 62 (2020) 355–364.
[47] P.K. Das, H.S. Behera, B.K. Panigrahi, A hybridization of an improved particle swarm optimization and gravitational search algorithm for multi-robot path planning, Swarm Evol. Comput. 28 (2016) 14–28.
[48] A. Koubaa, H. Bennaceur, I. Chaari, S. Trigui, A. Ammar, M.-F. Sriti, M. Alajlan, O. Cheikhrouhou, Y. Javed, Robot Path Planning and Cooperation, Springer, Cham, 2018.
[49] H.M. Choset (Ed.), Principles of Robot Motion: Theory, Algorithms, and Implementation, MIT Press, Cambridge, MA, 2005.
[50] R. Akay, M.Y. Yildirim, Multi-strategy and self-adaptive differential sine–cosine algorithm for multi-robot path planning, Expert Syst. Appl. 232 (2023) 120849.
[51] P.K. Das, P.K. Jena, Multi-robot path planning using improved particle swarm optimization algorithm through novel evolutionary operators, Appl. Soft Comput. 92 (2020) 106312.
[52] M.Z.B. Mohd Zain, J. Kanesan, J.H. Chuah, S. Dhanapal, G. Kendall, A multi-objective particle swarm optimization algorithm based on dynamic boundary search for constrained optimization, Appl. Soft Comput. 70 (2018) 680–700.
[53] X. Wang, S. Wang, X. Liang, D. Zhao, J. Huang, X. Xu, B. Dai, Q. Miao, Deep reinforcement learning: a survey, IEEE Trans. Neural Netw. Learn. Syst. 35 (2024) 5064–5078.
[54] Q. Zhou, Y. Lian, J. Wu, M. Zhu, H. Wang, J. Cao, An optimized Q-learning algorithm for mobile robot local path planning, Knowl. Based Syst. 286 (2024) 111400.
[55] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, D. Hassabis, Human-level control through deep reinforcement learning, Nature 518 (2015) 529–533.
[56] X. Ni, W. Hu, Q. Fan, Y. Cui, C. Qi, A Q-learning based multi-strategy integrated artificial bee colony algorithm with application in unmanned vehicle path planning, Expert Syst. Appl. 236 (2024) 121303.
[57] H. Kargaran, S. Yazdani, Set of Pareto solutions for optimum cascade problems using MOPSO algorithm, Results Eng. 16 (2022) 100625.
[58] N. Khodadadi, M. Ehteram, H. Karami, M.H. Nadimi-Shahraki, L. Abualigah, S. Mirjalili, Leader selection based multi-objective flow direction algorithm (MOFDA): a novel approach for engineering design problems, Results Eng. 25 (2025) 103670.
[59] Q.N.U. Islam, A. Ahmed, S.M. Abdullah, Optimized controller design for islanded microgrid using non-dominated sorting whale optimization algorithm (NSWOA), Ain Shams Eng. J. 12 (2021) 3677–3689.
[60] C. He, R. Cheng, L. Li, K.C. Tan, Y. Jin, Large-scale multiobjective optimization via reformulated decision variable analysis, IEEE Trans. Evol. Comput. 28 (2024) 47–61.
[61] S. Gupta, H. Abderazek, B.S. Yıldız, A.R. Yildiz, S. Mirjalili, S.M. Sait, Comparison of metaheuristic optimization algorithms for solving constrained mechanical design optimization problems, Expert Syst. Appl. 183 (2021) 115351.
[62] C.A.C. Coello, G.T. Pulido, M.S. Lechuga, Handling multiple objectives with particle swarm optimization, IEEE Trans. Evol. Comput. 8 (2004) 256–279.
[63] A. Got, D. Zouache, A. Moussaoui, MOMRFO: multi-objective manta ray foraging optimizer for handling engineering design problems, Knowl. Based Syst. 237 (2022) 107880.